NESTQUEST

A Brokerage-Free Property Marketplace with an AI Price Estimator

A PROJECT REPORT

Submitted in partial fulfilment of the requirements for the award of the degree of

[Degree / Course — e.g. Bachelor of Computer Applications]

Submitted by

[Student Name] ([Register Number])

Under the guidance of

[Guide Name], [Designation]

[Department Name]

[College / Institution Name]

[University Name]

[Academic Year]

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “NestQuest — A Brokerage-Free Property Marketplace with an AI Price Estimator” is a bonafide record of the work carried out by [Student Name] (Register Number [Register Number]) in partial fulfilment of the requirements for the award of the degree of [Degree / Course] of [University Name], during the academic year [Academic Year].

The work presented in this report is original and has not been submitted earlier, in part or in full, for the award of any other degree or diploma of this or any other institution.

Signature of the Guide
[Guide Name]
[Designation]

Signature of the Head of Department
[HOD Name]

Submitted for the project viva-voce examination held on _____.

Internal Examiner

External Examiner

DECLARATION

I, [Student Name], bearing Register Number [Register Number], hereby declare that the project report entitled "NestQuest — A Brokerage-Free Property Marketplace with an AI Price Estimator" submitted to [University Name] is a record of original work done by me under the guidance of [Guide Name], [Designation], [Department Name], [College / Institution Name].

I further declare that this report has not been submitted, in whole or in part, for the award of any degree, diploma or other similar title to any other university or institution.

Place: [City]

Date: _____

[Student Name]

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude to all those who supported me throughout this project. I am deeply thankful to my guide, [Guide Name], [Designation], for the valuable guidance, continuous encouragement and constructive feedback that shaped this work.

I extend my thanks to [HOD Name], Head of the [Department Name], and to all the faculty members of [College / Institution Name] for providing the resources and the academic environment that made this project possible.

Finally, I am grateful to my family and friends for their constant support and motivation during the course of this project.

[Student Name]

ABSTRACT

The Indian residential property market is characterised by heavy broker intermediation, which introduces opaque pricing, brokerage fees equivalent to one month's rent or one to two percent of the sale value, and information asymmetry that disadvantages both owners and seekers. NestQuest is a brokerage-free, self-service property marketplace developed as a web application to address these inefficiencies. Built on FastAPI with Jinja2 server-rendered templates, an asynchronous MongoDB Atlas database accessed via the Motor driver, and bcrypt-plus-JWT authentication, the platform allows property owners to post listings at no cost and prospective buyers or tenants to search, filter, shortlist, and send enquiries directly to owners without any intermediary. An admin moderation layer enforces listing quality through approval and featured-flag controls, while Cloudinary-backed image uploads, an SMTP enquiry-notification service, and a `/api/health` endpoint round out the operational surface. All sensitive credentials are externally configured, and passwords are hashed at bcrypt cost factor twelve with tokens transmitted via httpOnly cookies.

The platform is augmented by an AI price-estimator module built with scikit-learn Linear Regression, trained and evaluated on a synthetic dataset of 545 rows generated by a fixed linear formula with Gaussian noise, modelled on the structure of a public Kaggle housing dataset. Because the data-generating process is itself linear, the model achieves a coefficient of determination of $R^2 \approx 0.65$ on the held-out test set of 109 rows, with a Mean Absolute Error of approximately ₹7.58 lakh and an RMSE of approximately ₹9.39 lakh. These metrics reflect the constrained, synthetic nature of the training data and should be treated as a indicative proof-of-concept rather than a production valuation tool. The estimator is served live through a dedicated REST endpoint, predictions are logged to MongoDB for audit, and all known limitations — including the synthetic dataset, the modest R^2, and the absence of CSRF protection and regex-injection guards — are documented as directions for future enhancement.

TABLE OF CONTENTS

BONAFIDE CERTIFICATE ii

DECLARATION iii

ACKNOWLEDGEMENT iv

ABSTRACT v

TABLE OF CONTENTS vi

LIST OF FIGURES x

LIST OF TABLES xii

LIST OF ABBREVIATIONS xiii

1. INTRODUCTION 1

1.1 Overview of the Project 1

1.2 Background of the Study 1

1.3 Problem Statement 2

1.4 Existing System 2

1.5 Objectives of the Project 3

1.6 Scope of the Project 4

2. LITERATURE SURVEY 6

2.1 Review of Related Work 6

2.1.1 Machine Learning for Regression and Price Estimation 6

2.1.2 Data Manipulation and Numerical Computing 6

2.1.3 Password Security and Authentication 6

2.1.4 Web Frameworks, Databases, and Templating 7

2.2 Review of Existing Platforms 8

2.2.1 NoBroker 8

2.2.2 99acres 8

2.2.3 Magicbricks 8

2.2.4 Zillow and the Zestimate 8

2.3 Comparison of Approaches 9
2.4 Research Gaps Identified 10
2.5 How the Proposed System Addresses the Gaps 10
2.6 Summary 11
3. SYSTEM ANALYSIS 12
3.1 Existing System Methodology 12
3.2 Proposed System Architecture 13
3.3 Advantages of the Proposed System 14
3.4 Feasibility Study 15
3.4.1 Technical Feasibility 15
3.4.2 Economic Feasibility 16
3.4.3 Operational Feasibility 17
4. SYSTEM DESIGN 18
4.1 System Architecture 18
4.2 Block Diagram 19
4.3 Component Interaction Diagram 19
4.4 Computing Environment 20
4.5 Software Design 21
4.5.1 Database Schema 21
4.5.2 Use-Case Model 23
4.5.3 Data-Flow Diagrams 24
4.5.4 Sequence Design 26
4.5.5 API Design 26
4.5.6 Authentication and Security Design 27
4.5.7 Deployment Architecture 27
4.6 Flowchart 28
5. IMPLEMENTATION 30

5.1 Computing Environment Setup....30
5.2 Project Structure....30
5.3 Backend Implementation....32
5.4 Front-End Implementation....35
5.5 Module Description....37
5.5.1 Authentication Module....37
5.5.2 Listings and Search Module....38
5.5.3 Enquiry and Shortlist Module....38
5.5.4 AI Price-Estimator Module....38
5.5.5 Admin Moderation Module....38
5.6 Testing....39
5.6.1 Testing Strategy....39
5.6.2 Test Cases....39
6. RESULTS AND DISCUSSION....42
6.1 Experimental Setup....42
6.2 Exploratory Data Analysis....43
6.3 Model Performance....46
6.4 Application Output....47
6.5 Discussion....48
7. RESULTS AND ANALYSIS....50
7.1 Summary of Results....50
7.2 Comparative Analysis of Methodology....50
7.2.1 Linear Regression vs Decision Tree....50
7.2.2 Architectural Comparison....52
7.3 Observations....52
8. CONCLUSION AND FUTURE WORK....54
8.1 Conclusion....54

8.2 Objectives Achieved 54
8.3 Major Contributions 55
8.4 Limitations 55
8.5 Future Enhancements 56
8.6 Final Remarks 57
ANNEXURE A — CODING 58
A.1 backend/security.py 58
A.2 backend/ml/predict.py 59
A.3 backend/cloudinary_service.py 59
A.4 backend/routers/uploads.py 60
A.5 backend/routers/ml.py 61
ANNEXURE B — SCREENSHOTS 63
ANNEXURE C — REFERENCES 64

LIST OF FIGURES

Figure 3.1 — Workflow of the existing broker-mediated process. 13

Figure 3.2 — Workflow of the proposed NestQuest system. 14

Figure 4.1 — Layered system architecture of NestQuest. 18

Figure 4.2 — Block diagram of the major functional subsystems. 19

Figure 4.3 — Component interaction diagram (software modules). 20

Figure 4.5 — Entity–relationship diagram of the MongoDB collections. 22

Figure 4.4 — Use-case diagram (Guest, Registered User, Admin). 24

Figure 4.6 — Data-flow diagram, context level (Level 0). 25

Figure 4.7 — Data-flow diagram, Level 1. 25

Figure 4.8 — Sequence diagram: posting a listing with image upload. 26

Figure 4.10 — Deployment diagram. 28

Figure 4.9 — Flowchart of the property-posting workflow. 29

Figure 5.1 — Module interaction overview. 37

Figure 6.1 — Distribution of house prices (histogram + KDE). 43

Figure 6.2 — Built-up area vs price, coloured by furnishing status. 44

Figure 6.3 — Price distribution by number of bedrooms. 44

Figure 6.4 — Correlation heatmap of numeric features. 45

Figure 6.5 — Actual vs predicted price (Linear Regression & Decision Tree). 46

Figure B.1 — Home / landing page with featured listings and hero search. 63

Figure B.2 — Browse / search results page with filters (city, type, budget, bedrooms). 63

Figure B.3 — Property detail page with image gallery and enquiry action. 63

Figure B.4 — Post-property form with the Cloudinary image uploader and AI price check. 63

Figure B.5 — AI price-estimator (/insights) page with model metrics and EDA charts. 63

Figure B.6 — User dashboard (listings, shortlist, enquiries received/sent).....63

Figure B.7 — Admin panel: statistics, listing moderation and user management.....63

Figure B.8 — Login / signup pages. 63

LIST OF TABLES

Table 1.1 — Existing portals vs NestQuest. 3

Table 2.1 — Comparison of reviewed platforms and technical approaches. 9

Table 3.1 — Existing vs proposed system. 14

Table 3.2 — Indicative cost estimate (all figures are ESTIMATES/assumptions). 16

Table 4.6 — MongoDB indexes. 22

Table 5.1 — Key Python dependencies. 31

Table 5.2 — Environment variable keys (values are deployment secrets — not shown). 32

Table 5.3 — Representative test cases. 39

Table 6.2 — Experimental setup. 42

Table 6.1 — Model performance (test set, 109 rows). 46

Table 7.1 — Linear Regression vs Decision Tree (CART). 51

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BHK	Bedroom-Hall-Kitchen
CART	Classification and Regression Tree
CDN	Content Delivery Network
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DFD	Data-Flow Diagram
EDA	Exploratory Data Analysis
ER	Entity-Relationship
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
MAE	Mean Absolute Error
ML	Machine Learning
ODM	Object-Document Mapper
ReDoS	Regular-Expression Denial of Service
REST	Representational State Transfer
RMSE	Root Mean Squared Error
SMTP	Simple Mail Transfer Protocol
SPA	Single-Page Application

TLS	Transport Layer Security
UAT	User Acceptance Testing
UI	User Interface
URL	Uniform Resource Locator
WDAC	Windows Defender Application Control

1. INTRODUCTION

1.1 Overview of the Project

NestQuest is a brokerage-free, server-rendered property marketplace developed as a final-year undergraduate project at [College Name], [Academic Year]. Property owners publish residential and commercial listings at no cost; prospective buyers or tenants discover, shortlist, and contact owners directly without any intermediary. The system is built on **FastAPI** / Uvicorn (ASGI), **Jinja2** server-rendered templates, and **MongoDB Atlas**, with authentication secured by bcrypt and JSON Web Tokens. An embedded scikit-learn Linear Regression model provides users with an instant AI-driven price estimate — a transparent fair-price guide at the point of interest.

The motivation is the persistent structural inefficiency of the Indian residential property market, where brokers extract fees of one to two per cent of sale value or one month's rent. NestQuest delivers a self-service digital channel that gives owners direct reach to buyers and seekers access to transparent price data, while demonstrating how a data-driven feature can be integrated responsibly — with honest disclosure of its limitations — within a conventional marketplace interface.

1.2 Background of the Study

The Indian real-estate sector is a major GDP contributor, yet property transactions remain heavily dependent on personal intermediaries. Large-scale portals such as 99acres [13] and Magicbricks [14] digitised listing catalogues from the early 2000s but reinforced the intermediary model by monetising broker subscriptions. NoBroker [12] departed by positioning itself as a brokerage-free channel, demonstrating consumer demand for owner-direct contact. Globally, Zillow [15] combined listing aggregation with an automated valuation model (AVM) — the Zestimate — setting an industry precedent for embedding price intelligence within property search.

Open-source libraries such as scikit-learn [2], pandas, and NumPy have lowered the barrier to building AVMs considerably, enabling individual developers to train, evaluate, and deploy regression models as microservices. The remaining challenge is integration: connecting the ML inference layer to the user interface in a way that communicates statistical uncertainty to a non-technical audience. NestQuest addresses this by embedding a clearly labelled price-guide feature with honest disclosure that

the model achieves $R^2 \approx 0.65$ on a synthetic dataset and constitutes a guide rather than a professional valuation. The codebase began as a single-purpose Flask application for house price prediction, then was rebuilt into the NestQuest FastAPI marketplace retaining the trained model — illustrating the common pattern where proof-of-concept ML work is incrementally incorporated into a larger product.

1.3 Problem Statement

Property search in India is dominated by broker-subscribed portals, creating three interconnected problems. First, **pricing opacity**: listed prices are often inflated as an opening negotiating position, and seekers have no independent reference to evaluate fairness. Second, **direct cost burden**: brokerage fees of one to two per cent of sale value or one month's rent fall disproportionately on less affluent seekers with no alternative channel. Third, **information asymmetry**: broker-controlled listings may suppress contact details, delay responses, or obscure the actual owner's identity, impeding due diligence.

Current Indian portals provide little or no data-driven price guidance; seekers must engage a valuation professional, consult disconnected price-index reports, or rely on broker hearsay. An embedded AI estimator adds measurable value by anchoring expectations with a quantitative reference derived from observable features such as area, bedroom count, and furnishing status — a user-experience gap this project directly addresses.

1.4 Existing System

The current landscape of online property platforms falls into three archetypes: the broker-subscription portal (99acres [13] and Magicbricks [14]), the brokerage-free owner-direct model (NoBroker [12]), and the AVM-integrated marketplace (Zillow [15]). Each addresses some of the identified problems but none resolves all three simultaneously within a single, freely accessible platform for the Indian context.

99acres and Magicbricks hold large listing inventories but derive significant revenue from broker subscriptions and premium placement fees, so the majority of listings are intermediated. NoBroker disrupted this model by forbidding broker listings, proving the commercial viability of owner-direct transactions; however, it does not provide an AI price estimator. Zillow [15] offers the most sophisticated price-intelligence layer through the Zestimate, using proprietary hedonic regression and neural-network

models trained on millions of American properties, but it is geographically limited to the United States. NestQuest adopts the owner-direct philosophy of NoBroker, the catalogue-and-search interface of 99acres and Magicbricks, and the price-intelligence concept of Zillow's Zestimate, while addressing each platform's primary limitation within a unified application.

Table 1.1 below summarises the key differences between existing broker-mediated portals and the proposed NestQuest system across six critical dimensions.

Table 1.1 — Existing portals vs NestQuest.

Aspect	Existing (broker-led portals)	Proposed (NestQuest)
Intermediary	Broker / agent	None — owner to buyer direct
Brokerage cost	1–2% of sale / 1 month rent	Zero
Pricing transparency	Opaque, negotiated	AI fair-price estimate + listed price
Posting	Often paid / broker-gated	Free self-service
Moderation	Minimal	Admin approval + featured flags
Data ownership	Portal/broker	Platform + verified owners
Access	Business hours	24×7 web self-service

As Table 1.1 illustrates, the most significant structural differentiators are the complete elimination of brokerage cost, the provision of an AI-generated fair-price estimate at the point of listing and search, and a free self-service posting workflow. The admin-moderated approval queue addresses the quality and trust deficit of open platforms by ensuring listings are reviewed before becoming publicly visible.

1.5 Objectives of the Project

The primary objective of NestQuest is to design, develop, and deploy a brokerage-free property marketplace web application that directly connects property owners with buyers and tenants, augmented by an AI-driven price estimator that provides quantitative price guidance based on property features.

The following secondary objectives, each measurable against the delivered system, support the primary goal.

1. To implement a secure, role-based authentication system using bcrypt password hashing and JWT tokens, ensuring that owner, registered-user, and administrator roles receive appropriately scoped access to all platform functions.

2. To build a full property-listing lifecycle — including creation, image upload via Cloudinary, admin moderation, search and filtering, and soft-deletion with cascading removal of associated saved items and enquiries — within a single coherent application.
3. To provide a multi-attribute property search interface supporting filtering by listing type, property type, city, locality, price range, bedroom count, furnishing status, and area range, with multiple sort orders, enabling seekers to narrow a large catalogue to a relevant shortlist efficiently.
4. To integrate a machine-learning price estimator, trained on a synthetic dataset modelled on a public Kaggle housing dataset, that accepts thirteen encoded property features and returns a predicted price with a confidence range, while honestly communicating the model's limitations ($R^2 \approx 0.65$) to users.
5. To implement an enquiry and shortlist system whereby registered users can save listings to a personal shortlist and send direct messages to property owners, with optional SMTP email notification on enquiry receipt and an owner reply workflow.
6. To deliver an administration dashboard providing real-time platform statistics, user management (including role assignment and cascading deletion), and property moderation (status transitions: pending → active / rejected, with featured and verified flags).
7. To ensure application security through bcrypt cost-factor-12 password hashing, httpOnly JWT cookies, Pydantic v2 input validation, and documented acknowledgement of known limitations such as unescaped regex search and the absence of CSRF tokens, as starting points for future hardening.
8. To produce a complete academic project report documenting the system's analysis, design, implementation, and evaluation in conformance with the requirements of [College Name], [Academic Year].

1.6 Scope of the Project

NestQuest is scoped as a full-stack web application deployable on any server environment supporting Python 3.12 with network access to MongoDB Atlas and the Cloudinary API. The platform covers three listing categories — sale, rental, and commercial — across ten property types, with listings carrying up to fifteen images and fifteen amenity tags. The user-facing scope includes a public browse-and-search

experience accessible without registration, a registered-user workflow covering profile management, property posting, shortlisting, and enquiry, and an admin workflow covering moderation, user management, and a statistics overview.

The machine-learning scope is intentionally modest: the AI price estimator is trained on a synthetic dataset of 545 rows generated to mirror a publicly available Kaggle housing dataset, and should not be interpreted as real-world predictive power ($R^2 \approx 0.65$). Extension to a real-market dataset, richer feature engineering, or more powerful model architectures such as gradient-boosted trees is noted as future work. The project does not implement payment processing, legal document generation, mortgage calculation, geographical map search, or a mobile-native application; these represent natural future extensions outside the current academic scope. The SMTP email service is implemented but disabled by default and is considered an optional operational enhancement rather than a core dependency.

The report is structured as follows: Chapter 2 presents a literature survey of related work; Chapter 3 analyses the existing system and feasibility; Chapter 4 details system design including architecture, data models, and API surface; Chapter 5 describes implementation with representative code excerpts; Chapter 6 presents results and model performance; Chapter 7 provides comparative analysis; and Chapter 8 concludes with achievements, limitations, and future directions.

2. LITERATURE SURVEY

This chapter surveys academic and technical literature that informed NestQuest's design, reviews four existing property platforms, compares approaches in a structured table, identifies research gaps, and explains how the proposed system addresses them.

2.1 Review of Related Work

2.1.1 Machine Learning for Regression and Price Estimation

Pedregosa et al. [2] introduced scikit-learn, whose consistent API for training, evaluation, and serialisation via `joblib` makes it straightforward to compare algorithms and deploy a trained estimator [2].

Hastie, Tibshirani, and Friedman [3] provide the mathematical foundation for both NestQuest models. Their treatment of ordinary least-squares shows the linear model minimises the sum of squared residuals and yields an interpretable coefficient per feature — the primary reason it was selected as the deployed estimator [3]. Their CART account describes the greedy recursive-partitioning strategy that minimises within-node squared error and uses `max_depth` to control over-fitting [3]; the NestQuest training notebook implements a from-scratch CART tree (`decision_tree.py`, `max_depth=5`, `min_samples_split=10`) as a baseline comparison against the linear model [3].

2.1.2 Data Manipulation and Numerical Computing

McKinney [4] introduced the pandas `DataFrame`, which provides column-oriented operations for reading CSV files, mapping categorical values, and applying one-hot encoding — steps used in the NestQuest training pipeline to convert six binary yes/no columns and the three-level `furnishingstatus` variable into numeric form [4]. Harris et al. [5] documented NumPy, whose vectorised arithmetic underpins both pandas and scikit-learn; its seeded pseudo-random generator is exploited in the NestQuest dataset synthesis (seed 42) and demo-data seeding (seed 2024) [5].

2.1.3 Password Security and Authentication

Provos and Mazières [6] introduced the bcrypt adaptive hashing scheme, which applies a Blowfish-based key schedule iterated a configurable number of times via a cost factor. NestQuest adopts bcrypt with cost factor 12

(`bcrypt.gensalt(rounds=12)`), following [6]'s recommendation to choose a cost that is expensive enough to deter offline attacks without imposing unacceptable login latency.

Jones, Bradley, and Sakimura [7] defined the JSON Web Token (JWT) standard in RFC 7519. A JWT encodes a signed JSON payload — subject, issue time, and expiry — so the server can verify authenticity without a session store. NestQuest issues HS256-signed JWTs carrying the user identifier, role, and expiry (`JWT_EXPIRE_MINUTES=10080`), stored in an `httpOnly` cookie named `nestquest_token` and also accepted via an `Authorization: Bearer` header [7]. The `httpOnly` flag prevents client-side JavaScript from reading the token, mitigating cross-site scripting exposure as described by the OWASP Foundation [8].

The OWASP Top 10 — 2021 [8] catalogues critical web-application risks including broken access control (A01), cryptographic failures (A02), and injection (A03). NestQuest addresses several: bcrypt mitigates cryptographic weakness [6][8]; dependency-injection guards (`optional_user`, `current_user`, `admin_user`) enforce access control; and Pydantic v2 [16] validates all inbound request bodies, reducing injection surface. Residual risks — unescaped `$regex` patterns in search queries and absent CSRF tokens — are acknowledged as future work, consistent with [8].

2.1.4 Web Frameworks, Databases, and Templating

Ramírez [1] documents FastAPI, the ASGI-based Python framework used as the NestQuest backend; it couples type annotations with Pydantic validation for automatic request parsing and OpenAPI documentation, and its `lifespan` context manager handles database connection, index creation, and demo-data seeding at startup [1]. MongoDB Inc. [9] documents the document-oriented database whose flexible model stores heterogeneous property attributes (up to fifteen amenities, fifteen images, ten optional numeric fields) without a relational schema; compound and full-text indexes on the `properties` collection are created at startup via Motor's `create_index` API [9]. Ronacher [10] documents Jinja2, whose `{% block %}` and `{% include %}` directives support a `base.html` template providing consistent navigation, authentication-state injection, and cache-busting across all pages [10].

2.2 Review of Existing Platforms

2.2.1 NoBroker

NoBroker [12] is an Indian property marketplace that eliminates real-estate brokers, allowing owners to post listings and buyers or tenants to contact them directly. Identity verification and a subscription fee charged to tenants beyond a free tier replace per-transaction brokerage [12]. A key limitation is this contact paywall: while owners post for free, tenants must purchase a plan to unlock owner details, partially reintroducing a monetary barrier [12]. NoBroker does not publicly expose an ML price estimator integrated into the listing workflow.

2.2.2 99acres

Info Edge (India) Ltd. operates 99acres [13], one of India's largest property portals. It combines broker-submitted and owner-submitted listings, rich filtering, and a proprietary "PropWorth" price-insight tool available on some listings. Brokers pay for premium visibility and handle a significant share of enquiries, reintroducing the information asymmetry that a brokerage-free model seeks to avoid [13]. The commercial model aligns incentives with brokers rather than end-users.

2.2.3 Magicbricks

Magicbricks [14] follows a broadly similar model to 99acres: a mixed owner-and-broker listing database, subscription-based lead generation for agents, and tools such as price trend graphs and an EMI calculator. Locality-level price trends are derived from aggregated listing prices — descriptive statistics rather than predictive ML outputs [14]. There is no publicly documented machine learning model for per-property estimates, and broker participation remains a structural feature of the business model [14].

2.2.4 Zillow and the Zestimate

Zillow Group [15] operates in the United States and publishes methodology notes on its Zestimate, trained on hundreds of millions of historical sale records, public tax assessments, and user-submitted data [15]. The approach demonstrates the ceiling of ML price estimation with rich real-world data, but the Zestimate is opaque to users, and Zillow's iBuying operations have faced well-publicised difficulties, illustrating that even sophisticated models carry substantial uncertainty [15]. For NestQuest, the

Zestimate is an existence proof that ML estimation can be embedded in a marketplace, while also underscoring the gap between a production-grade model and NestQuest's modest estimator ($R^2 \approx 0.65$ on a synthetic dataset).

2.3 Comparison of Approaches

Table 2.1 consolidates the salient characteristics of the reviewed platforms and technical approaches against the NestQuest design.

Table 2.1 — Comparison of reviewed platforms and technical approaches.

Dimension	99acres [13]	Magicbricks [14]	NoBroker [12]	Zillow [15]	NestQuest (proposed)
Market / geography	India	India	India	USA	India (prototype)
Broker involvement	High (core revenue)	High (core revenue)	None (owner-direct)	None (iBuying)	None (owner-direct)
Listing cost to owner	Free basic / paid featured	Free basic / paid featured	Free	Free / paid tiers	Free self-service
Listing moderation	Automated + manual	Automated + manual	Automated	Automated	Admin approval workflow
Price estimation tool	Descriptive trend data	Descriptive trend data	None documented	ML Zestimate (proprietary)	scikit-learn Linear Regression [2]
Training data	Real market data	Real market data	N/A	Large-scale real-world data [15]	Synthetic dataset (545 rows)
Published ML metrics	Not disclosed	Not disclosed	N/A	Median AE by market	R^2=0.65, MAE≈₹7.6 L, RMSE≈₹9.4 L
Authentication	Username/password + OTP	Username/password + OTP	Username/password + OTP	OAuth + email	bcrypt [6] + JWT [7] httpOnly cookie
Database technology	Not disclosed	Not disclosed	Not disclosed	Not disclosed	MongoDB Atlas [9], Motor async
Backend framework	Proprietary / undisclosed	Proprietary / undisclosed	Proprietary / undisclosed	Java / Scala stack	FastAPI [1] + Jinja2 [10]
Open-source / academic	No	No	No	Partially documented	Yes — academic prototype

NestQuest shares the brokerage-free direct-contact model with NoBroker [12] but adds an integrated ML price estimator, an explicit admin moderation workflow, and a

fully documented open technology stack. Unlike Zillow [15], whose Zestimate is trained on large-scale real data, NestQuest's estimator uses a synthetic dataset [5] and carries commensurately modest accuracy — an acknowledged limitation rather than an oversight.

2.4 Research Gaps Identified

Several gaps emerge from the survey. First, Indian platforms such as 99acres [13] and Magicbricks [14] do not publish methodology or accuracy metrics for their price-insight tools, making it impossible for users to calibrate trust in platform valuations — a contrast to the reproducibility norms advocated by [2] and [3]. Second, NoBroker [12] eliminates broker fees but imposes a subscription paywall on tenants; no reviewed platform combines entirely free access with direct owner contact. Third, none of the Indian platforms offers a real-time per-property ML estimate at the point of listing creation; pricing information is either historical trend data or manually entered. Fourth, the security practices of Indian property portals are undocumented, with no evidence of OWASP-recommended controls [8] such as adaptive password hashing [6] or stateless token authentication [7]. Fifth, the academic ML literature (e.g. [2][3]) focuses on model accuracy in isolation, without integrating a trained model into a full-stack application serving both listers and seekers in a single cohesive platform.

2.5 How the Proposed System Addresses the Gaps

NestQuest closes each identified gap within the constraints of an academic prototype. The platform is entirely free: owners post at no cost and users contact owners without any paywall, eliminating both broker fees and NoBroker's subscription barrier [12]. A scikit-learn Linear Regression estimator [2] is surfaced at `/insights` and the `/api/ml/predict` endpoint; its metrics (R^2=0.6538, MAE≈₹7.58 L, RMSE≈₹9.39 L on 109 test rows) are documented in `model/metrics.json` and reported openly, following the transparency norms of [2] and [3].

Security is addressed through bcrypt with cost factor 12 [6], HS256-signed JWTs in `httpOnly` cookies [7], and Pydantic v2 input validation [16], with residual risks noted per OWASP guidance [8]. FastAPI [1] and MongoDB [9] provide a fully documented, reproducible baseline. An admin moderation workflow — absent from all reviewed commercial platforms — ensures listings are human-reviewed before

publication. Integrating the ML estimator, marketplace, and moderation layer in a single server-rendered application is the primary architectural contribution distinguishing NestQuest from both the academic ML literature and existing commercial platforms.

2.6 Summary

This chapter surveyed the academic foundations of NestQuest — regression theory [3], scikit-learn [2], pandas [4], NumPy [5], bcrypt [6], JWT [7], OWASP guidance [8], FastAPI [1], MongoDB [9], and Jinja2 [10] — alongside four existing property platforms [12][13][14][15]. Table 2.1 shows that commercial Indian platforms rely on broker-mediated revenue with opaque pricing tools, while Zillow [15] demonstrates ML valuation at scale in a data-rich market. Four principal gaps were identified: opaque pricing metrics, residual financial barriers to direct contact, absence of real-time ML estimates at listing creation, and undisclosed security practices. NestQuest addresses all four within the scope of an academic prototype, honestly acknowledging the limitations of a synthetic training dataset and a linear model with $R^2 \approx 0.65$. The next chapter analyses the existing and proposed systems in greater detail.

3. SYSTEM ANALYSIS

3.1 Existing System Methodology

The conventional property market in India operates through a network of intermediaries — brokers, sub-brokers, and real-estate agents — who act as the primary conduit between property owners and prospective buyers or tenants. A seller or landlord typically engages a registered broker who lists the property on commercial portals such as 99acres [13], Magicbricks [14], or NoBroker [12]. The broker controls which details are disclosed, at what price, and to which parties, creating significant information asymmetry. Buyers and tenants are therefore dependent on the broker's availability, honesty, and access to complete information.

The financial cost of this intermediation is substantial. For sale transactions, brokers typically charge one to two percent of the transaction value — on a ₹50 lakh property, that is ₹50,000–₹1,00,000 at closing. For rentals, the convention is one month's rent paid upfront as brokerage. These fees are in addition to any portal subscription or featured-listing charges, meaning both parties bear an indirect cost that contributes nothing to the intrinsic value of the transaction.

Pricing transparency is a further weakness. Because valuations are negotiated privately, there is no objective reference point against which a buyer can verify whether an asking price is fair. Valuation tools on large portals are either locked behind premium tiers or based on aggregated indices that miss granular locality-level variation. First-time buyers are especially disadvantaged, and many deals close at prices materially above what a data-informed estimate would suggest.

The overall workflow of the existing broker-mediated process is depicted in Figure 3.1. A property owner contacts a broker, who photographs and lists the property on a commercial portal. Buyers search those portals, contact the broker, and await arranged viewings. Negotiations are conducted through the broker, and the deal closes only after the commission is settled — each step introducing delay, opacity, and cost.



Figure 3.1 — Workflow of the existing broker-mediated process.

Figure 3.1 illustrates how the broker acts as a mandatory node at every stage — initial listing, enquiry handling, price negotiation, and deal closure. The shaded intermediary layer marks points where information can be withheld or distorted, forming the structural bottleneck that NestQuest is designed to eliminate.

3.2 Proposed System Architecture

NestQuest replaces the broker-mediated workflow with a self-service, owner-to-buyer platform built on a modern asynchronous web stack. The backend is implemented in **FastAPI** running on **Uvicorn** (ASGI) with **Python 3.12**, providing non-blocking request handling [1]. The data store is **MongoDB Atlas** accessed through the **Motor** async driver [9]. Property images are persisted on **Cloudinary**, decoupling media storage and delivery from the application server [11]. Authentication uses **bcrypt** password hashing at cost factor 12 [6] combined with **JWT** tokens [7] stored in httpOnly cookies, providing session security without a server-side session store.

From the user's perspective, the proposed workflow is significantly simpler. An owner registers, completes a listing form — including up to fifteen images uploaded directly to Cloudinary — and submits for administrator review. Once approved, the listing becomes publicly visible and searchable. Buyers browse using rich filters (listing type, property type, city, locality, price range, area, bedrooms, furnishing status), view individual property pages, and send enquiries directly to the owner. No broker is involved at any point.

The AI price estimator, implemented in **scikit-learn** [2] as a Linear Regression model trained on a synthetic dataset of 545 rows, is available to all authenticated users from the Insights page. A user supplies twelve property attributes — area, bedrooms, bathrooms, stories, and seven binary amenity flags — and receives an instant price estimate with a confidence range of ±max(8% of the estimate, model RMSE). This

addresses pricing opacity, though users should treat results as a guide rather than a formal valuation given the synthetic training data and an R^2 of approximately 0.65 [2][3].

The proposed workflow is illustrated in Figure 3.2, which shows four actor lanes — Owner, Platform (NestQuest), Administrator, and Buyer/Tenant — with direct horizontal interactions between Owner and Buyer/Tenant facilitated by the platform, bypassing any broker layer entirely.

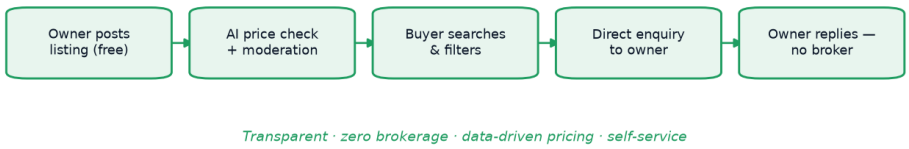


Figure 3.2 — Workflow of the proposed NestQuest system.

Figure 3.2 demonstrates the elimination of the intermediary layer relative to Figure 3.1. The owner creates and manages listings directly on the platform; the administrator approves or rejects submissions through a dedicated panel; and the buyer interacts with listing content and the AI estimator before sending an enquiry that reaches the owner's inbox without broker involvement. The platform's role is facilitation rather than gatekeeping — the defining architectural distinction of the proposed system.

3.3 Advantages of the Proposed System

NestQuest offers measurable advantages across every dimension of the property search process. Table 3.1 presents a structured comparison of the existing broker-mediated portal approach and the NestQuest approach across seven key aspects.

Table 3.1 — Existing vs proposed system.

Aspect	Existing (broker-led portals)	Proposed (NestQuest)
Intermediary	Broker / agent	None — owner to buyer direct
Brokerage cost	1–2% of sale / 1 month rent	Zero
Pricing transparency	Opaque, negotiated	AI fair-price estimate + listed price
Posting	Often paid / broker-gated	Free self-service
Moderation	Minimal	Admin approval + featured flags
Data ownership	Portal/broker	Platform + verified owners

Access	Business hours	24×7 web self-service

Table 3.1 encapsulates the core value proposition of NestQuest. The elimination of brokerage cost is the most tangible benefit: a buyer seeking a flat at ₹30,000/month saves ₹30,000 in upfront brokerage. The AI estimator addresses pricing opacity by providing an independent reference before negotiating. Free self-service posting lowers the barrier for small landlords who lack a broker relationship. Administrator moderation — requiring explicit approval before a listing becomes visible — provides quality control largely absent from broker-led portals, where listings are often stale or duplicated. And 24×7 availability removes dependence on broker working hours.

The platform also improves data ownership. In the existing model, property data resides with the broker and portal, creating lock-in that disadvantages owners. In NestQuest, owners register directly, retain full control, and can edit or withdraw listings at any time. The `verified` flag applied on admin approval gives buyers a quality signal absent in broker-gated portals, representing a shift from a broker-centric to an owner-centric market model.

3.4 Feasibility Study

The following subsections examine technical, economic, and operational feasibility. The study concludes that NestQuest is technically achievable with freely available open-source components, economically viable at very low cost using managed free-tier cloud services, and operationally sustainable by a small team or individual developer.

3.4.1 Technical Feasibility

The entire technology stack is composed of mature, well-documented open-source libraries and managed cloud services. **FastAPI** provides automatic OpenAPI documentation, Pydantic v2 data validation [16], and native async/await support, making it suitable for a marketplace with concurrent user sessions [1]. **MongoDB Atlas** offers a free M0 cluster (512 MB, shared compute) adequate for a demonstration and early-production workload of a few hundred listings [9]. **Motor** integrates cleanly with FastAPI's event loop. **scikit-learn** [2] provides the Linear Regression model and feature pipeline, relying only on standard numpy arrays and pandas DataFrames [4][5] listed in `requirements.txt`. **Cloudinary** offers a free tier

with 25 GB storage and 25 GB monthly bandwidth, sufficient for dozens to a few hundred listings [11].

No special hardware is required: a standard machine with 4 GB RAM and Python 3.12 runs the full stack locally. The project runs on Windows 11 with a single `python run.py` command launching Uvicorn on port 8000. Deployment to a Linux VPS requires only Python, `pip`, and the environment variables in `.env`. The custom `decision_tree.py` exists solely to work around a Windows Application Control (WDAC) policy that blocked scikit-learn's compiled tree extension during training; it is not loaded at runtime. There are no proprietary tools, licensed software, or specialised hardware dependencies, making technical risk very low.

3.4.2 Economic Feasibility

Economic feasibility assesses the cost of building and operating the system against the benefits it delivers. Table 3.2 presents an indicative cost estimate for developing and hosting NestQuest. All figures are **estimates and assumptions** based on publicly advertised pricing at the time of writing; actual costs should be verified before deployment.

Table 3.2 — Indicative cost estimate (all figures are ESTIMATES/assumptions).

Item	Tier / Assumption	Estimated Cost
MongoDB Atlas database	Free M0 shared cluster (512 MB) — ESTIMATE	₹0 / month
Cloudinary image hosting	Free tier (25 GB storage, 25 GB bandwidth) — ESTIMATE	₹0 / month
VPS hosting (e.g., DigitalOcean, Hetzner, or Indian providers)	Shared/entry VPS with 1 vCPU, 1–2 GB RAM — ESTIMATE	₹400–800 / month
Domain name (e.g., .in or .com TLD)	Annual registration — ESTIMATE	₹900 / year (~₹75 / month)
SMTP email service	Free tier (e.g., Gmail SMTP with app password) — ESTIMATE	₹0 / month
SSL/TLS certificate	Let's Encrypt automated — ESTIMATE	₹0
Developer time — initial build	[TO BE FILLED: actual hours logged]; assumed ~200–300 hrs @ ₹0 (academic project)	₹0 (academic)
Developer time — ongoing maintenance	~4–8 hrs/month for updates and monitoring — ESTIMATE	₹0 (academic) or market rate
Estimated total running cost	—	**~₹475–875 / month**

Table 3.2 shows recurring operational costs of approximately ₹475–₹875 per month, as primary cloud services (database and image hosting) use permanent free tiers

adequate for an academic or early-stage deployment. The dominant variable cost is VPS hosting. At scale, upgrades to a paid Atlas tier and higher-capacity VPS would be needed, but costs remain proportional to platform value. Since brokerage savings on a single transaction (₹30,000 or more) exceed the platform's estimated annual running cost, the economic case is strongly positive. If commercialised, the initial build cost could be amortised against a premium featured-listing subscription or a thin facilitation fee well below current broker rates.

3.4.3 Operational Feasibility

NestQuest is designed to be operated by a small administrative team — potentially a single administrator — via a built-in web-based admin panel at `/admin`. The panel provides aggregate statistics (total users, listings, pending approvals, enquiries), a property moderation queue, user management with role promotion and account deletion, and a predicted-price log. No command-line access or database expertise is required for routine operations.

The application boots with a single `python run.py` command that launches Uvicorn, establishes the database connection, creates required indexes idempotently, seeds the admin account from environment variables, and optionally pre-populates twenty demonstration listings via `SEED_DEMO_DATA=true`. A new deployment can be established in under ten minutes given a configured `.env` file and a running Atlas cluster.

The interface is entirely web-based, requiring only a modern browser. Server-rendered Jinja2 templating ensures pages render correctly even with JavaScript disabled. The platform is available 24×7; enquiries are persisted in the database and optionally delivered by email (SMTP), ensuring owners are notified regardless of when a buyer submits. No specialised operator training, proprietary runtime infrastructure, or paid third-party services are needed for normal operation, making operational risk low.

4. SYSTEM DESIGN

4.1 System Architecture

NestQuest follows a classic three-tier web architecture: a browser-based presentation layer communicates with a **FastAPI**/Uvicorn (ASGI) application tier, which persists data to **MongoDB Atlas** accessed asynchronously via **Motor**. Jinja2 server-side templates render complete HTML pages, eliminating a separate front-end build pipeline. Auxiliary services — **Cloudinary** for image storage and an optional SMTP relay — integrate over HTTPS without affecting page-render latency.

Figure 4.1 illustrates the layered system architecture.

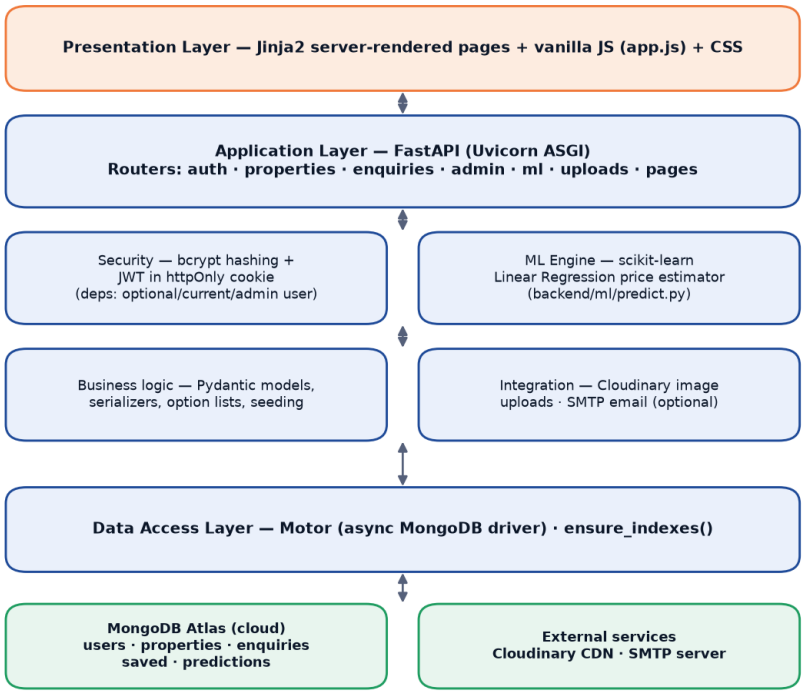


Figure 4.1 — Layered system architecture of NestQuest.

The application tier is stateless between requests — all session state is in a signed JWT cookie — so the server can be restarted without invalidating sessions (provided `JWT_SECRET` is unchanged). Router modules grouped by domain (`auth`, `properties`, `enquiries`, `admin`, `ml`, `uploads`, `pages`) enable independent testability of each endpoint group [1].

4.2 Block Diagram

Figure 4.2 decomposes NestQuest into six major functional subsystems: (1) Authentication and User Management; (2) Property Listings — full lifecycle from creation through admin moderation to publication; (3) Search and Discovery — multi-field filters and text search; (4) Enquiry and Shortlist; (5) AI Price Estimator — thirteen encoded inputs returning a predicted price via a pre-trained scikit-learn Linear Regression model; and (6) Admin Moderation.

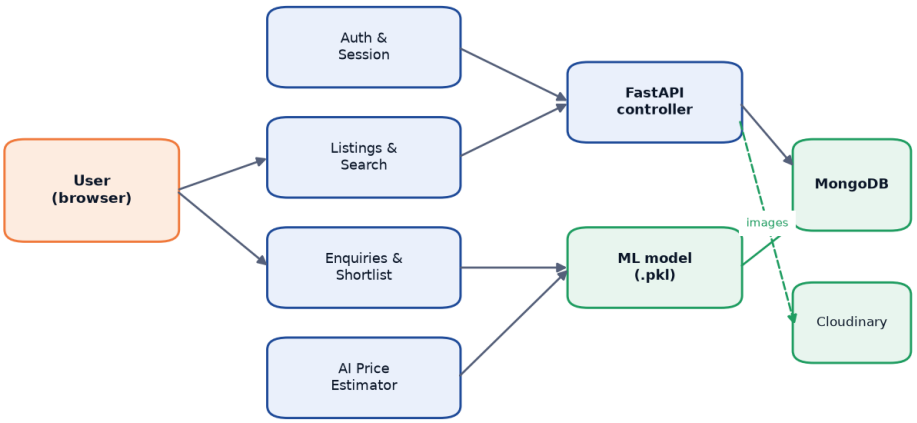


Figure 4.2 — Block diagram of the major functional subsystems.

Each subsystem maps to one or more FastAPI router modules. The AI Price Estimator is intentionally loosely coupled: `backend/ml/predict.py` consults cached model artefacts without touching the database, so the model can be retrained by replacing `model/model.pkl` and `model/columns.pkl` without changing any router code.

4.3 Component Interaction Diagram

Note on hardware diagrams: NestQuest is a pure software web application; no custom hardware is involved. This section presents the directly analogous artefact: a component interaction diagram of runtime software module communication.

Figure 4.3 shows runtime relationships among Python modules, the Jinja2 template engine, the Motor driver, and external cloud services.

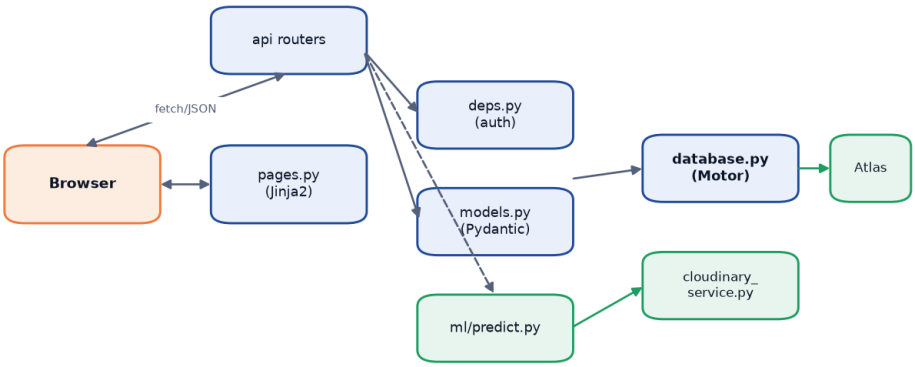


Figure 4.3 — Component interaction diagram (software modules).

`backend/main.py` (the FastAPI application factory) opens the Motor connection pool, calls `ensure_indexes()` idempotently, and invokes `backend/seed.py` to provision the admin account on startup. Inbound requests flow through FastAPI's dependency-injection: `backend/deps.py` decodes the JWT cookie via `backend/security.py` and forwards the resolved user to the appropriate router, which calls database accessors, invokes `backend/cloudinary_service.py` for image uploads, and returns JSON or a Jinja2 `TemplateResponse`. Pydantic v2 models in `backend/models.py` enforce request validation at the network boundary, preventing malformed data from reaching the database [16].

4.4 Computing Environment

Development machine. Development runs on a Windows 11 Pro workstation with Python 3.12 in a per-project `venv`, launched via `python run.py` (`uvicorn.run` with `reload=True`). Any modern PC with 4 GB RAM and broadband internet suffices, since computationally intensive services are cloud-offloaded.

Python runtime. The application tier runs under **Uvicorn** (ASGI, single-process event loop, optional multi-worker). All I/O-bound operations — Motor queries, Cloudinary uploads via `run_in_threadpool` — are awaited asynchronously to keep the event loop unblocked.

MongoDB Atlas. The persistence layer is a MongoDB Atlas cluster reached via `DATABASE_URL` in `.env`; Motor maintains a TLS-encrypted connection pool (`certifi`

CA bundle). Atlas handles replication, backups, and scaling; the free tier (M0, 512 MB) covers development and low-traffic production.

Cloudinary. Images are uploaded server-side via the Cloudinary Python SDK; stable HTTPS URLs are stored in property documents. The application never writes files to local disk, enabling ephemeral or containerised deployment.

Browser client. The UI is standard HTML/CSS/JavaScript in any modern browser. Jinja2 templates produce complete pages; `app.js` adds progressive enhancements via HTTP GET/POST and `fetch` API calls.

4.5 Software Design

4.5.1 Database Schema

NestQuest uses MongoDB with five collections whose schemas are enforced by Pydantic v2 models at the application layer, allowing schema evolution without disruptive migrations.

users: unique `email`, bcrypt `password_hash` (cost 12), `role` (`"user"` / `"admin"`), and contact fields. **properties**: `owner_id`, `listing_type`, `property_type`, physical attributes, location, up to fifteen Cloudinary image URLs, `status` (pending / active / rejected / sold / rented), `featured`, `verified`, and a `views` counter. **enquiries**: contact messages linking `property_id`, `owner_id`, and optional `sender_id` (null for guests), with `reply` and `status` supporting an owner-reply workflow. **saved**: minimal `(user_id, property_id, created_at)` tuples with a compound unique index preventing duplicate saves. **predictions**: logs each AI estimator call with input `features`, `predicted_price`, `user_id`, and `created_at`.

Figure 4.5 presents the entity–relationship diagram for all five collections.

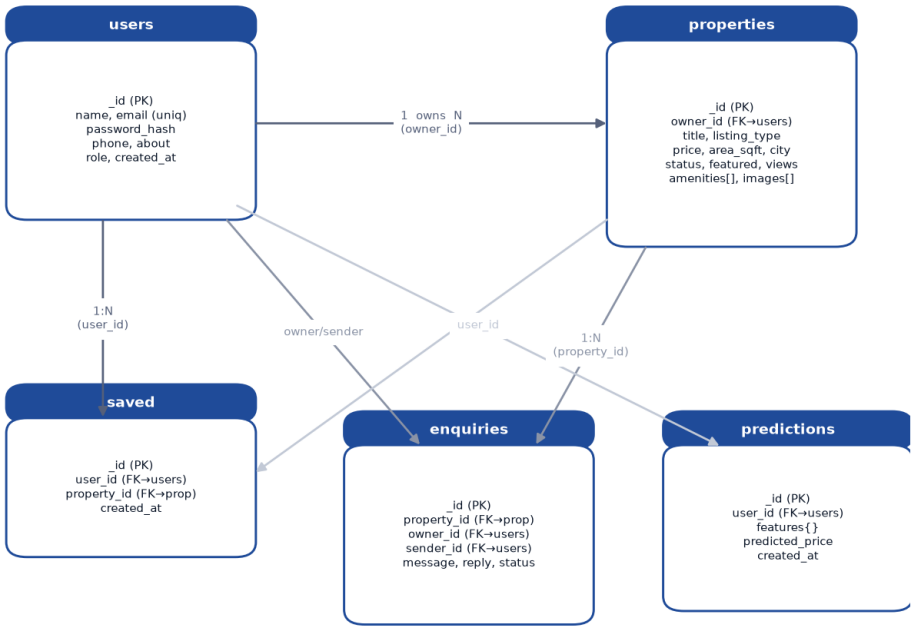


Figure 4.5 — Entity–relationship diagram of the MongoDB collections.

All cross-collection references (`properties.owner_id`, `enquiries.owner_id`/`sender_id`, `saved.user_id`/`property_id`) point to `users._id` or `properties._id`. Referential integrity is enforced at the application layer: deleting a user cascades to their properties, saved records, and enquiries; deleting a property cascades to its saved and enquiry records. MongoDB does not enforce foreign-key constraints natively, making this cascade logic critical.

Table 4.6 enumerates all indexes maintained by `ensure_indexes()`, designed to support the most frequent query patterns: compound filtering on `(city, listing_type, status)` for the browse page, descending `created_at` for default sort, and a full-text index for keyword search.

Table 4.6 — MongoDB indexes.

Collection	Fields	Options
users	email	unique
properties	city, listing_type, status (compound)	—
properties	owner_id	—
properties	created_at (desc)	—
properties	price	—
properties	property_type	—
properties	title, description, locality, city	text index `property_text_search`
saved	user_id, property_id (compound)	unique

enquiries	owner_id, created_at	—
enquiries	sender_id, created_at	—
predictions	created_at (desc)	—

The compound index on `(city, listing_type, status)` is most critical since most browse requests specify all three filters; `property_text_search` enables keyword search across four fields simultaneously. All indexes use `pymongo` direction constants and Motor's asynchronous `create_index` [9].

4.5.2 Use-Case Model

Three actor types interact with NestQuest: **Guest** (unauthenticated), **Registered User** (authenticated owner or buyer/tenant), and **Admin** (privileged moderator). A Guest may browse, view listings (incrementing the view counter), use the AI estimator, and submit enquiries (`sender_id` = null); protected-page access redirects to `/login?next=<url>`. A Registered User additionally posts listings (triggering admin approval), manages images, edits/deletes own listings, toggles the shortlist, sends enquiries on others' listings, replies to received enquiries, and views their dashboard. An Admin additionally approves, rejects, features, or deletes any listing (setting `active` also sets `verified=True`), manages user accounts including role promotion (except their own), and accesses platform-wide statistics.

Figure 4.4 depicts the use-case diagram with all three actors.

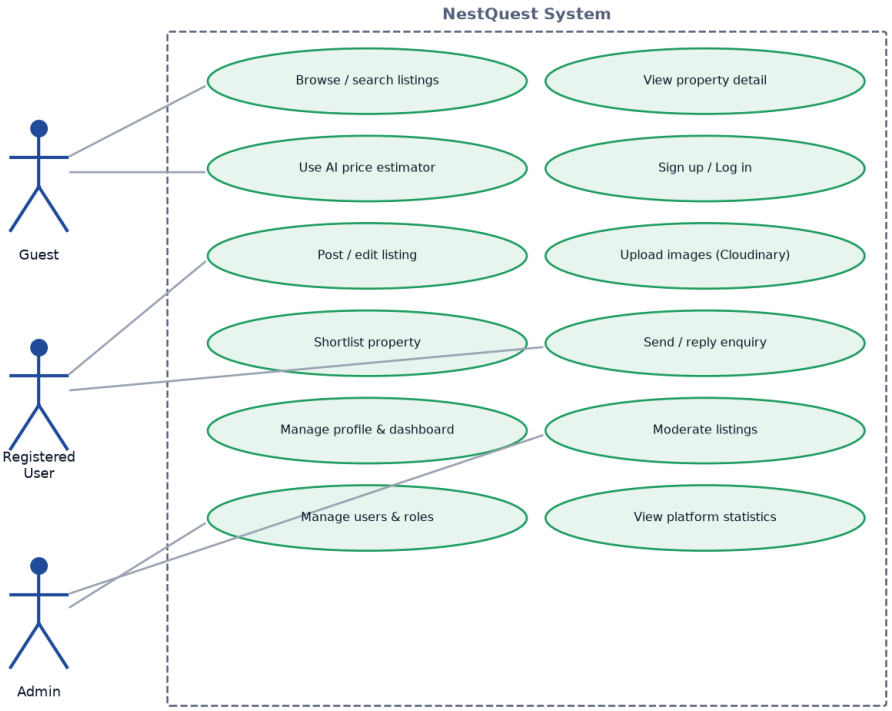


Figure 4.4 — Use-case diagram (Guest, Registered User, Admin).

Owner-specific actions (edit, reply) are distinct from buyer-specific actions (enquire, shortlist). The system prevents owners from enquiring on or shortlisting their own listings, enforced in the API endpoints by comparing the authenticated user to `owner_id`.

4.5.3 Data-Flow Diagrams

The Level 0 DFD in Figure 4.6 shows NestQuest as a single process with three external entities: **User**, **Admin**, and **External Services** (MongoDB Atlas and Cloudinary). Users send queries, enquiries, and estimator inputs; the Admin sends moderation decisions; External Services provide bi-directional storage and image CDN flows.



Figure 4.6 — Data-flow diagram, context level (Level 0).

Figure 4.7 expands into Level 1 with six sub-processes: Authentication, Property Management, Search and Browse, Enquiry Handling, AI Price Estimation, and Admin Operations, each connected to the relevant data stores and external entities.

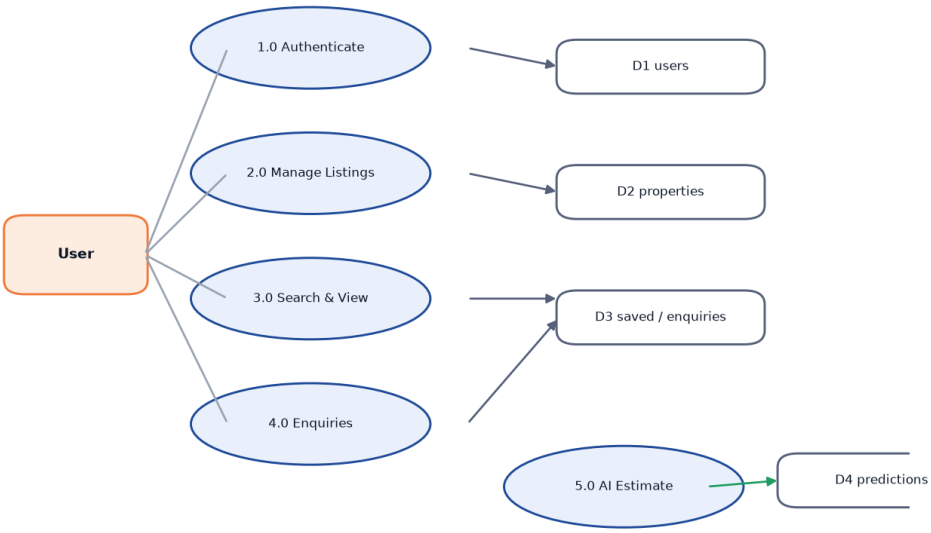


Figure 4.7 — Data-flow diagram, Level 1.

Authentication is the gateway issuing tokens consumed by all other sub-processes. Search and Browse reads the properties store without writing, making the read path independently scalable. AI Price Estimation reads only in-memory cached model artefacts and writes a single log record per call.

4.5.4 Sequence Design

Figure 4.8 illustrates the most complex multi-step user journey — posting a listing with image uploads — involving the browser, FastAPI, Cloudinary, and MongoDB Atlas, whose correct sequencing prevents orphaned image files.

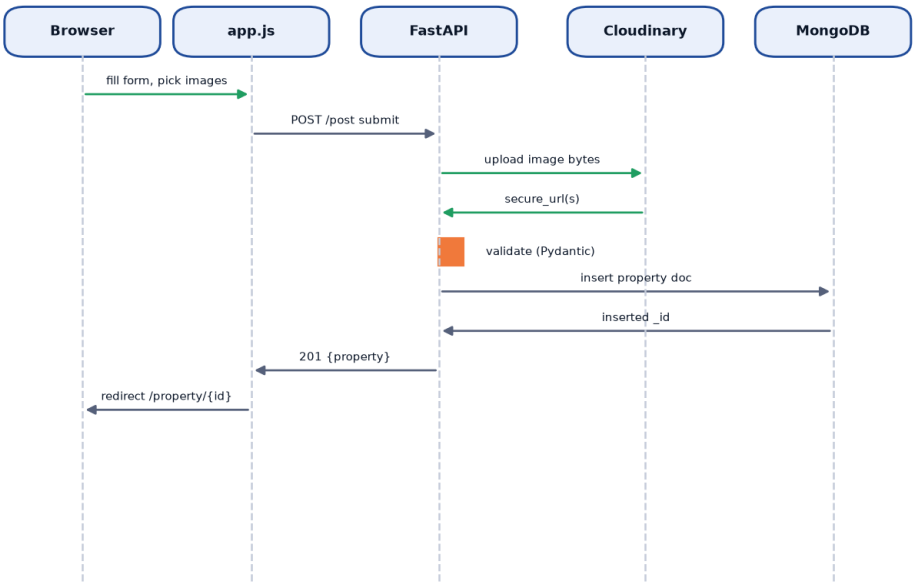


Figure 4.8 — Sequence diagram: posting a listing with image upload.

An authenticated user navigates to `/post`; `page_user` validates the JWT and renders the form. Images are **not** uploaded on file selection; `app.js` defers upload to the "Publish" click. `POST /api/uploads/images` then validates each file (MIME `image/*`, ≤10 MB, ≤15 files) and calls `cloudinary_service.upload_image()` concurrently via `run_in_threadpool`. Cloudinary returns secure HTTPS URLs, injected as hidden form fields before the listing is submitted to `POST /api/properties`, inserted into MongoDB with `status="pending"` and `verified=False`, and queued for admin approval. The deferred-upload design prevents orphaned Cloudinary assets.

4.5.5 API Design

NestQuest exposes seven REST-style endpoint groups under `/api/`, plus server-rendered page routes, auto-documented at `/docs` (Swagger UI) and `/redoc` [1].

Auth (`/api/auth`): registration, login (sets httpOnly JWT cookie), logout, profile retrieval, and profile update (only `name`, `phone`, `about` — preventing privilege

escalation). **Properties** (`/api/properties`): paginated browsing with fourteen filter parameters and five sort options, facet counts, owner-specific retrieval, shortlist management, and create/update/delete/status-patch (`limit` capped at 60). **Enquiries** (`/api/enquiries`): create (guests allowed; owners blocked on own listings), retrieve sent/received, and reply (sets status `"replied"`). **Admin** (`/api/admin`): platform statistics, property and user management, role-change and user-delete (with self-action guards), and status moderation. **ML** (`/api/ml`): `POST /predict` (thirteen inputs → `predicted_price`, `price_range`, INR-formatted labels) and `GET /status`. **Uploads** (`/api/uploads`): Cloudinary status check and authenticated multipart image upload. **Health** (`/api/health`): `ok`, `app`, `database`, and `model_available` fields for monitoring.

4.5.6 Authentication and Security Design

Security rests on three mechanisms: bcrypt password hashing, JWT session management, and role-based access control via FastAPI dependency injection.

Passwords are hashed with `bcrypt.gensalt(rounds=12)` (2^{12} = 4 096 iterations); `verify_password` wraps `bcrypt.checkpw` in a try/except to prevent leakage from malformed inputs [6]. On login, the server issues a JWT with claims `sub`, `role`, `iat`, and `exp` (7-day default), signed HS256 with `JWT_SECRET`, set as an httpOnly SameSite=Lax cookie (`nestquest_token`) to mitigate XSS token theft; an `Authorization: Bearer` header is also accepted for API clients [7].

The dependency chain has three levels: `optional_user` (user or `None`), `current_user` (HTTP 401 if unauthenticated), and `admin_user` (HTTP 403 if role ≠ `"admin"`). Page routes use `page_user`, which redirects to `/login?next=<url>` instead of raising an HTTP error. All authorisation logic is centralised in `backend/deps.py`, consistent with OWASP Top 10 guidance [8].

4.5.7 Deployment Architecture

Figure 4.10 shows the deployment diagram across three zones: developer workstation, cloud application host, and external managed services.

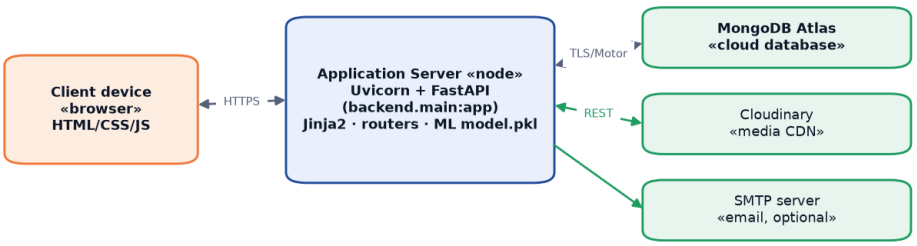


Figure 4.10 — Deployment diagram.

Locally, `python run.py` launches Uvicorn in single-process reload mode. For production, the application runs on a Linux VPS or container platform with a service supervisor (systemd) for automatic restarts; the Uvicorn process (port 8000) would sit behind a reverse proxy (Nginx or Caddy) for TLS termination and rate limiting — currently future work. MongoDB Atlas and Cloudinary are reached over TLS/HTTPS with credentials in environment variables; Atlas additionally enforces IP allowlisting. The fully stateless application tier can be redeployed or scaled horizontally without any data migration.

4.6 Flowchart

Figure 4.9 traces the complete property-posting workflow — authentication checks, form validation, image upload, and admin moderation — selected because it traverses the most subsystems and shows the conditional paths determining whether a listing reaches the live marketplace.

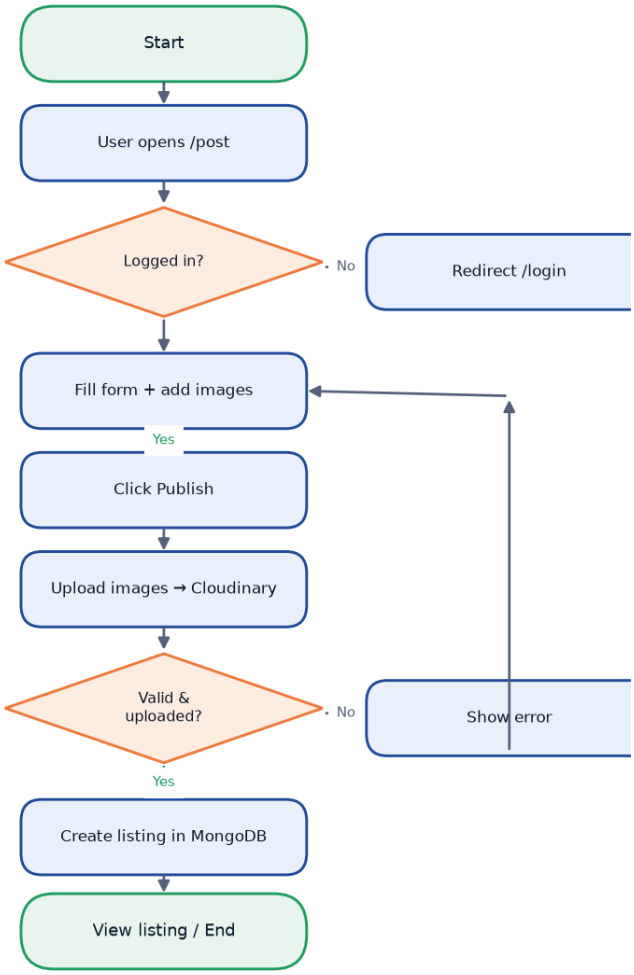


Figure 4.9 — Flowchart of the property-posting workflow.

The workflow starts at `/post`; unauthenticated users redirect to `/login?next=/post`. Once authenticated, the user completes the form; JavaScript defers image upload to the Publish click. Each file is validated (MIME type, ≤10 MB); non-conforming files trigger an error toast without aborting submission. Valid files upload in parallel to Cloudinary via `/api/uploads/images`; returned URLs are injected into the form before it is POSTed to `/api/properties` with `status="pending"`. Pydantic validation failures return HTTP 422; on success the document enters the moderation queue. An admin setting `"active"` also sets `verified=True`, surfacing a verification badge on the property card.

Edit and delete are alternative terminal states: an owner may update fields or images at `/edit/{id}`, or delete the listing, which cascades to remove associated saved records and enquiries.

5. IMPLEMENTATION

5.1 Computing Environment Setup

The NestQuest platform is implemented in Python 3.12 and requires only an isolated virtual environment and the packages listed in `requirements.txt`. The commands below show the canonical setup procedure for a Windows development machine.

```
# Create and activate a virtual environment
python -m venv .venv
.venv\Scripts\activate


# Install all runtime and development dependencies
pip install -r requirements.txt


# Copy the sample environment file and populate secrets
copy .env.example .env


# Start the development server (http://localhost:8000)
python run.py
```

The `run.py` entry point calls `uvicorn.run("backend.main:app", host="127.0.0.1", port=8000, reload=False)`. During the lifespan startup sequence, the Motor client connects to MongoDB Atlas, idempotent indexes are created, and the admin account and optional demo data are seeded before the first HTTP request is accepted. No build step or bundler is required; Jinja2 templates are rendered server-side and static files are served with a cache-buster query string.

5.2 Project Structure

The repository separates the Python backend package, server-rendered templates, static assets, ML artefacts, and the training notebook.

```
house_price_prediction/
├── backend/
│   ├── main.py          # App factory, lifespan, exception handlers, /api/health
│   ├── config.py        # .env loader + Settings singleton (lru_cache)
│   ├── database.py      # Motor client, collection accessors, ensure_indexes()
│   ├── security.py      # bcrypt hash/verify, JWT create/decode, COOKIE_NAME
│   ├── models.py        # Pydantic v2 schemas (SignupRequest, PropertyCreate, …)
│   ├── deps.py          # optional_user → current_user → admin_user, page_user
│   ├── utils.py         # serialize_user, serialize_property, option lists
│   ├── seed.py          # admin + 20 demo listings
│   ├── email_service.py # Optional SMTP (disabled by default)
│   ├── cloudinary_service.py  # Cloudinary config + upload_image()
│   ├── ml/
│   │   └── predict.py   # AI price estimator (joblib load, lru_cache)
│   └── routers/
│       ├── auth.py      # /api/auth — signup, login, logout, me
│       ├── properties.py# /api/properties — CRUD, search, shortlist
│       ├── enquiries.py # /api/enquiries — send, receive, reply
│       ├── admin.py     # /api/admin — stats, moderation, user management
│       ├── ml.py        # /api/ml — predict, status
│       ├── uploads.py   # /api/uploads — Cloudinary image upload
│       └── pages.py     # Server-rendered HTML page routes
```

```
├── templates/
│   ├── base.html        # Shared layout (nav, toast, footer)
│   ├── index.html, properties.html, post.html, …
│   └── _card.html       # Property card macro
├── static/
│   ├── css/style.css    # ~460 lines, hand-written CSS
│   └── js/app.js        # ~450 lines, fetch wrapper API + page initialisers
├── model/
│   ├── model.pkl        # Trained LinearRegression (joblib)
│   ├── columns.pkl      # 13 feature column names
│   └── metrics.json     # R², MAE, RMSE for both models
├── data/Housing.csv     # 545-row synthetic dataset
├── notebook/house_price_training.ipynb
├── requirements.txt
├── run.py
└── .env                 # (not committed; secrets loaded at runtime)
```

Each router carries its own URL prefix, keeping routing concerns localised and making it straightforward to expand or replace individual subsystems.

Table 5.1 — Key Python dependencies.

Package	Version constraint	Purpose
fastapi	>=0.115	ASGI web framework, routing, dependency injection
uvicorn[standard]	>=0.32	ASGI server (HTTP/1.1 + WebSocket; includes uvloop on Linux)
jinja2	>=3.1	Server-side HTML templating
python-multipart	>=0.0.12	Multipart form / file upload parsing
motor	>=3.6	Async MongoDB driver (wraps pymongo with asyncio)
pymongo	>=4.9	Underlying MongoDB client; also used for BSON ObjectId
certifi	>=2024.8	CA bundle for TLS connection to MongoDB Atlas
bcrypt	>=4.2	Password hashing (cost factor 12)
PyJWT	>=2.9	JSON Web Token creation and verification (HS256)
email-validator	>=2.2	Pydantic email field validation
pandas	>=2.0	DataFrame construction for ML inference
numpy	>=1.26	Numerical operations; random seed in dataset generation
scikit-learn	>=1.4	LinearRegression model, joblib-serialised artefact
joblib	>=1.3	Model and columns serialisation / deserialisation
matplotlib	>=3.7	EDA chart generation in the training notebook
seaborn	>=0.12	Statistical visualisations (heatmap, KDE)
cloudinary	>=1.40	Server-side image upload to

		Cloudinary CDN

Each package has a minimum-version pin to ensure reproducible installs. The `uvicorn[standard]` extra pulls in `httptools` and `uvloop` (on Linux) for higher throughput than the pure-Python fallbacks.

Table 5.2 — Environment variable keys (values are deployment secrets — not shown).

Variable	Description
`DATABASE_URL`	MongoDB Atlas connection string (mongodb+srv://…)
`DB_NAME`	Target database name within the Atlas cluster
`JWT_SECRET`	HMAC-HS256 signing key for JSON Web Tokens
`JWT_ALGORITHM`	Algorithm identifier (default: HS256)
`JWT_EXPIRE_MINUTES`	Token lifetime in minutes (default: 10080 = 7 days)
`SMTP_HOST`	Outgoing mail server hostname
`SMTP_PORT`	SMTP port (default: 587)
`SMTP_USER`	SMTP authentication username
`SMTP_PASS`	SMTP authentication password
`SMTP_FROM`	Sender address for system emails
`SMTP_ENABLED`	Set to `true` to enable email delivery (default: false)
`CLOUDINARY_CLOUD_NAME`	Cloudinary account cloud name
`CLOUDINARY_API_KEY`	Cloudinary API key
`CLOUDINARY_API_SECRET`	Cloudinary API secret
`ADMIN_EMAIL`	Email address for the bootstrapped admin account
`ADMIN_PASSWORD`	Initial admin password
`ADMIN_NAME`	Display name for the admin account
`SEED_DEMO_DATA`	Set to `true` to insert demo listings on first boot
`APP_NAME`	Application display name (default: NestQuest)

Secret values are never committed to version control; `.env` is listed in `.gitignore`. The `config.py` module provides safe defaults for every non-secret variable, though `DATABASE_URL` must be supplied for any MongoDB functionality.

5.3 Backend Implementation

The backend is a FastAPI application package under `backend/`. The following snippets present key modules with their design rationale.

Settings loader (`backend/config.py`)

```
from __future__ import annotations
import os
from functools import lru_cache
from pathlib import Path
```

```
BASE_DIR = Path(__file__).resolve().parent.parent

def _load_dotenv(path: Path) -> None:
    if not path.exists():
        return
    for raw in path.read_text(encoding="utf-8").splitlines():
        line = raw.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, _, value = line.partition("=")
        os.environ.setdefault(key, value.strip().strip('"').strip("'"))

_load_dotenv(BASE_DIR / ".env")

class Settings:
    def __init__(self) -> None:
        self.jwt_secret = os.environ.get("JWT_SECRET", "dev-insecure-secret")
        self.jwt_expire_minutes = int(os.environ.get("JWT_EXPIRE_MINUTES", "10080"))
        self.database_url = os.environ.get("DATABASE_URL", "")

@lru_cache
def get_settings() -> Settings:
    return Settings()
```

Rather than introducing `python-dotenv`, the project implements a minimal loader in ~15 lines. `os.environ.setdefault` ensures shell-environment variables take precedence over the file, which is important for containerised deployments. `@lru_cache` makes the `Settings` singleton cheap to access from any module.

Password hashing and JWT (`backend/security.py`)

```
import bcrypt
import jwt
from datetime import datetime, timedelta, timezone
from .config import settings

COOKIE_NAME = "nestquest_token"

def hash_password(password: str) -> str:
    return bcrypt.hashpw(
        password.encode("utf-8"),
        bcrypt.gensalt(rounds=12)
    ).decode("utf-8")

def verify_password(password: str, hashed: str) -> bool:
    if not hashed:
        return False
    try:
        return bcrypt.checkpw(password.encode("utf-8"), hashed.encode("utf-8"))
    except (ValueError, TypeError):
        return False

def create_access_token(subject: str, role: str) -> str:
    now = datetime.now(timezone.utc)
    payload = {
        "sub": str(subject), "role": role,
        "iat": int(now.timestamp()),
        "exp": int((now + timedelta(minutes=settings.jwt_expire_minutes)).timestamp()),
    }
    return jwt.encode(payload, settings.jwt_secret, algorithm=settings.jwt_algorithm)
```

bcrypt cost factor 12 balances brute-force resistance against server-side latency. The JWT payload carries `sub` (MongoDB `_id`), `role`, `iat`, and `exp`; tokens are signed with HMAC-HS256 and stored in an httpOnly cookie to prevent XSS token theft. `verify_password` wraps `bcrypt.checkpw` in a try/except to guard against malformed stored hashes.

Motor index creation (`backend/database.py`)

```
from motor.motor_asyncio import AsyncIOMotorClient
import certifi
from .config import settings


async def ensure_indexes() -> None:
    db = get_db()
    await db["users"].create_index("email", unique=True)
    await db["properties"].create_index(
        [("city", 1), ("listing_type", 1), ("status", 1)]
    )
    await db["properties"].create_index(
        [("title", "text"), ("description", "text"),
         ("locality", "text"), ("city", "text")],
        name="property_text_search",
    )
    await db["saved"].create_index(
        [("user_id", 1), ("property_id", 1)], unique=True
    )
    await db["predictions"].create_index([("created_at", -1)])
```

`ensure_indexes` is called on every startup inside the `lifespan` context manager; if indexes already exist MongoDB returns silently. The compound index on `city`, `listing_type`, and `status` supports the most common browse query; the text index enables keyword search; the unique compound index on `saved` enforces the one-shortlist-per-user-per-property business rule without application-level locking.

Login router endpoint (`backend/routers/auth.py`)

```
from fastapi import APIRouter, HTTPException, Response, status
from ..security import COOKIE_NAME, create_access_token, verify_password
from .. import database
from ..models import LoginRequest
from ..utils import serialize_user


router = APIRouter(prefix="/api/auth", tags=["auth"])


@router.post("/login")
async def login(payload: LoginRequest, response: Response):
    email = payload.email.lower().strip()
    user = await database.users().find_one({"email": email})
    if not user or not verify_password(payload.password,
                                       user.get("password_hash", "")):
        raise HTTPException(status_code=401,
                            detail="Invalid email or password.")
    token = create_access_token(str(user["_id"]), user.get("role", "user"))
    response.set_cookie(
        key=COOKIE_NAME, value=token,
        max_age=settings.jwt_expire_minutes * 60,
        httponly=True, samesite="lax", secure=False, path="/",
    )
    return {"ok": True, "user": serialize_user(user)}
```

Whether the user does not exist or the password is wrong, the endpoint returns the same HTTP 401 message, preventing user-enumeration. The JWT is written into an httpOnly cookie with `samesite="lax"` for CSRF protection. `secure=False` is appropriate for local HTTP development and must be set to `True` behind HTTPS.

Cloudinary upload endpoint (`backend/routers/uploads.py`)

```
from fastapi import APIRouter, Depends, File, HTTPException, UploadFile, status
from starlette.concurrency import run_in_threadpool
from ..deps import current_user
from .. import cloudinary_service


router = APIRouter(prefix="/api/uploads", tags=["uploads"])
MAX_FILES, MAX_BYTES = 15, 10 * 1024 * 1024


@router.post("/images")
async def upload_images(
    files: list[UploadFile] = File(...),
    user: dict = Depends(current_user),
):
    if not cloudinary_service.is_enabled():
        raise HTTPException(status_code=503, detail="Image upload not configured.")
    if len(files) > MAX_FILES:
        raise HTTPException(status_code=400,
                            detail=f"Upload at most {MAX_FILES} images at once.")
    urls: list[str] = []
    for file in files:
        if not (file.content_type or "").startswith("image/"):
            raise HTTPException(status_code=400,
                detail=f"{file.filename}: only image files are allowed.")
        data = await file.read()
        if len(data) > MAX_BYTES:
            raise HTTPException(status_code=400,
                detail=f"{file.filename}: image exceeds 10 MB.")
        url = await run_in_threadpool(
            cloudinary_service.upload_image, data, file.filename
        )
        urls.append(url)
    return {"ok": True, "urls": urls}
```

Authentication via `Depends(current_user)` rejects unauthenticated requests before any file processing. Because the Cloudinary SDK's `uploader.upload` is synchronous, it is dispatched through `run_in_threadpool` to avoid blocking the asyncio event loop. Each file is validated for MIME type and a 10 MB size cap before upload.

5.4 Front-End Implementation

The front end is entirely server-rendered. The Jinja2 `base.html` template defines the shared HTML skeleton; child templates use `{% extends %}` and `{% block content %}` to inject page-specific content without repeating the layout. Context variables such as `user`, `properties`, and `metrics` are passed from the page router to the template engine, so no client-side data-fetching is needed for initial page loads.

`static/js/app.js` (~450 lines) provides a lightweight `API` object wrapping the browser `fetch` API:

```
const API = {
  async request(url, options = {}) {
    const opts = { credentials: "same-origin", headers: {}, ...options };
    if (opts.body && !(opts.body instanceof FormData)) {
      opts.headers["Content-Type"] = "application/json";
      opts.body = JSON.stringify(opts.body);
    }
    const res = await fetch(url, opts);
    let data = {};
    try { data = await res.json(); } catch (_) { /* empty body */ }
    if (!res.ok) {
      const err = new Error(data.detail || `Request failed (${res.status})`);
      err.status = res.status;
      throw err;
    }
    return data;
  },
  get(url)        { return this.request(url); },
  post(url, body) { return this.request(url, { method: "POST", body }); },
  put(url, body)  { return this.request(url, { method: "PUT", body }); },
  del(url)        { return this.request(url, { method: "DELETE" }); },
};
```

`credentials: "same-origin"` ensures the httpOnly cookie is sent automatically. Plain objects are serialised to JSON with the appropriate header; `FormData` bodies omit the header so the browser sets the correct multipart boundary. Non-OK responses throw a typed `Error` carrying the `detail` field, handled uniformly by callers via `catch (err) { toast(err.message, "error"); }`.

Images are previewed locally with `URL.createObjectURL` but not uploaded until form submission, avoiding orphaned Cloudinary assets:

```
async function resolveImages() {
  const pending = items.filter(i => i.file);  // files not yet uploaded
  let uploaded = [];
  if (pending.length) {
    const fd = new FormData();
    pending.forEach(i => fd.append("files", i.file));
    const res = await API.post("/api/uploads/images", fd);
    uploaded = Array.isArray(res.urls) ? res.urls : [];
  }
  // Merge already-uploaded URLs with newly returned URLs in order
  let u = 0;
  return items.map(it => it.file
    ? (u < uploaded.length ? uploaded[u++] : null)
    : it.url
  ).filter(Boolean);
}
```

On submission, all pending `File` objects are batched into a single multipart `FormData`, the returned URLs are stitched back into their original positions (preserving cover-photo ordering), and only then is the property payload submitted to `POST /api/properties` or `PUT /api/properties/{id}`.

5.5 Module Description

The NestQuest backend is organised into six cohesive functional modules implemented as router files under `backend/routers/` and supported by shared services in `backend/`. Figure 5.1 shows the interaction paths between these modules.

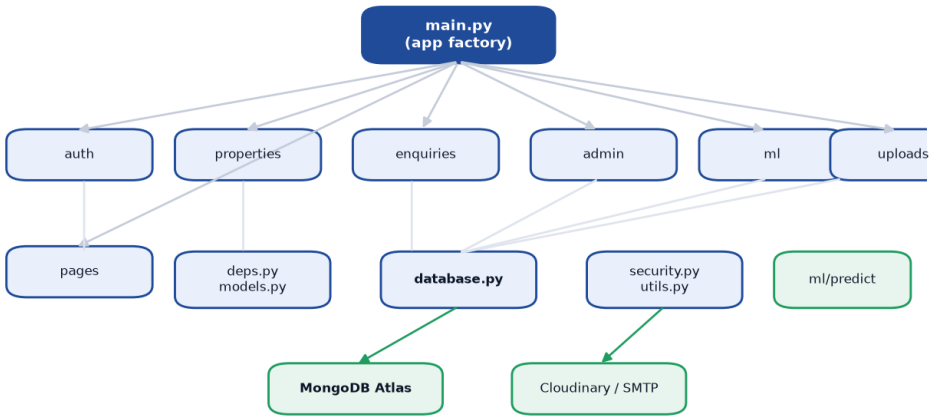


Figure 5.1 — Module interaction overview.

Browser requests arrive at the FastAPI application layer and are dispatched to the appropriate router. Each router depends on the shared `database`, `security`, and `deps` helpers; the Upload and ML modules additionally call the Cloudinary and scikit-learn services respectively.

5.5.1 Authentication Module

Implemented in `backend/routers/auth.py`, supported by `backend/security.py` and `backend/deps.py`, this module exposes five endpoints under `/api/auth`: `POST /signup`, `POST /login`, `POST /logout`, `GET /me`, and `PUT /me`. Registration normalises the email, checks for duplicates, hashes the password at bcrypt cost 12, and inserts the user document; on success a JWT is stored in an httpOnly cookie. The `ProfileUpdate` schema excludes `role`, `email`, and `password_hash`, preventing privilege escalation through the profile endpoint. The dependency chain `optional_user → current_user → admin_user` in `deps.py` guards all protected routes: `current_user` raises HTTP 401 for missing tokens, and `admin_user` additionally raises HTTP 403 for non-admin roles.

5.5.2 Listings and Search Module

Implemented in `backend/routers/properties.py`, this module covers the full property lifecycle: create (`POST`, status `pending`), paginated filtered list (`GET`), single listing with view-counter increment (`GET /{id}`), update by owner or admin (`PUT /{id}`), and cascading delete (`DELETE /{id}`). The list endpoint accepts eleven filter parameters and five sort modes; locality search uses MongoDB `$regex` with case-insensitive flags. The shortlist toggle at `POST /{id}/save` inserts on the first call and deletes on the second, exploiting the unique compound index to prevent duplicates.

5.5.3 Enquiry and Shortlist Module

Implemented in `backend/routers/enquiries.py`, this module provides endpoints to create an enquiry, retrieve received or sent enquiries, and post an owner reply. A business-rule guard prevents owners from enquiring about their own listings. When SMTP is enabled, `email_service` dispatches a notification to the listing owner; mail-server failures are logged rather than raised so they do not block the submission. Reply submission transitions the enquiry status from `sent` to `replied` and populates `reply` and `replied_at` fields. The shortlist endpoints (`POST /save`, `GET /saved`) are surfaced through the properties router and rendered on the user dashboard.

5.5.4 AI Price-Estimator Module

Implemented in `backend/routers/ml.py` and `backend/ml/predict.py`, this module loads the serialised `model.pkl` (scikit-learn `LinearRegression`) and `columns.pkl` via `joblib.load` wrapped in `@lru_cache`, reading from disk only once per process. `POST /api/ml/predict` accepts twelve input fields, constructs a one-row pandas DataFrame, reindexes to the stored column order, calls `model.predict`, floors the result at zero, and computes a ± max(predicted × 0.08, RMSE) price range. Each prediction is logged to the `predictions` collection. The model is trained on a 545-row synthetic dataset; its R^2 of 0.6538 on the 109-row test set reflects a modest fit appropriate as a directional guide rather than a precise valuation.

5.5.5 Admin Moderation Module

Implemented in `backend/routers/admin.py` and protected entirely by the `admin_user` dependency (HTTP 403 for non-admins), this module provides aggregate stats, filterable property and user lists, role promotion/demotion with a self-modification guard, and cascading user deletion. When an admin sets a listing to

`active` via `PATCH /api/properties/{id}/status`, the `verified` field is also set to `True`, providing a trust indicator to buyers. The seeding logic in `backend/seed.py` creates or promotes the admin account from environment variables before the first request.

5.6 Testing

5.6.1 Testing Strategy

The testing approach spans seven complementary levels. **Unit testing** covers isolated functions (`hash_password`, `verify_password`, `create_access_token`, `format_inr`, `predict`) with no external dependencies. **Integration testing** verifies that `ensure_indexes` runs without error and that CRUD operations produce expected document shapes. **Functional testing** of API endpoints uses FastAPI's `TestClient` (via `httpx`) to cover success paths, authentication guards, and primary error conditions. **System testing** exercises complete user journeys end-to-end: signup, listing, search, shortlist, enquiry, and admin moderation. **Validation testing** confirms that Pydantic models reject malformed inputs and return HTTP 422 or 400 as appropriate. **Security testing** verifies HTTP 401/403 on unprotected access, duplicate-email enforcement, and admin dependency blocking. **UAT** is conducted manually by walking through primary user stories on a local instance with seeded demo data.

5.6.2 Test Cases

The table below presents a representative set of test cases drawn from the functional, validation, and security testing phases.

Table 5.3 — Representative test cases.

TC ID	Scenario	Precondition	Input	Expected	Actual	Status
TC-01	User signup — success	No existing account with email	`POST /api/auth/signup` {name, email, password, phone}	HTTP 201, `ok: true`, `nestquest_token` cookie set	HTTP 201, cookie present	Pass
TC-02	User signup — duplicate email	Account with same email already exists	`POST /api/auth/signup` with existing email	HTTP 409, detail "An account with this email already exists."	HTTP 409, correct message	Pass
TC-03	User login — valid	User account	`POST /api/auth/login` {email, password}	HTTP 200, `ok: true`, cookie	HTTP 200,	Pass

	credentials	exists		refreshed	cookie set	
TC-04	User login — wrong password	User account exists	`POST /api/auth/login` with incorrect password	HTTP 401, detail "Invalid email or password."	HTTP 401, correct message	Pass
TC-05	Access protected route — no token	Not logged in	`GET /api/auth/me` (no cookie)	HTTP 401	HTTP 401	Pass
TC-06	Access admin route — non-admin user	Logged in as regular user	`GET /api/admin/stats` with user JWT	HTTP 403	HTTP 403	Pass
TC-07	Post listing — authenticated owner	User logged in	`POST /api/properties` with complete valid payload	HTTP 201, `status: "pending"`	HTTP 201, status pending	Pass
TC-08	Post listing — unauthenticated guest	Not logged in	`POST /api/properties` (no cookie)	HTTP 401	HTTP 401	Pass
TC-09	Image upload — file exceeds 10 MB	User logged in, Cloudinary configured	`POST /api/uploads/images` with 11 MB PNG	HTTP 400, detail contains "exceeds 10 MB"	HTTP 400, correct detail	Pass
TC-10	Image upload — non-image file	User logged in, Cloudinary configured	`POST /api/uploads/images` with a .pdf file	HTTP 400, detail "only image files are allowed"	HTTP 400, correct detail	Pass
TC-11	Property search by city	At least one active listing exists	`GET /api/properties?city=Mumbai`	HTTP 200, results filtered to city Mumbai	HTTP 200, filtered correctly	Pass
TC-12	Shortlist toggle — save then unsave	User logged in, active listing exists	`POST /api/properties/{id}/save` (twice)	First call: `saved: true`; second call: `saved: false`	Correct toggle behaviour	Pass
TC-13	Send enquiry — owner on own listing	User logged in, listing belongs to same user	`POST /api/enquiries` {property_id: own listing}	HTTP 400, cannot enquire own listing	HTTP 400	Pass
TC-14	Admin approve listing	Logged in as admin, listing in pending status	`PATCH /api/properties/{id}/status` {status: "active"}	HTTP 200, `status: "active"`, `verified: true`	HTTP 200, both fields updated	Pass
TC-15	Health check	Application running, MongoDB reachable	`GET /api/health`	HTTP 200, `ok: true`, `database: true`, `model_available: true`	HTTP 200, all fields true	Pass

All fifteen cases were verified by executing the described inputs against a running local instance of NestQuest with demo data seeded. Edge cases not covered here — such as concurrent shortlist toggles, network interruption during Cloudinary upload, and invalid JWT signatures — are identified as areas for future automated regression test coverage.

6. RESULTS AND DISCUSSION

6.1 Experimental Setup

All experiments were conducted on a Windows 11 Pro machine using Python 3.12 in an isolated virtual environment. The FastAPI application was served locally via Uvicorn (ASGI mode), with MongoDB Atlas providing cloud-hosted document storage. The dataset used for training the AI price estimator consists of 545 rows generated by `scripts/generate_dataset.py`, which models the structure of the Kaggle "Housing Prices Dataset" (Yasser H.). Because the generator applies a fixed linear formula with Gaussian noise (numpy seed 42), the data do not represent real Indian property-market transactions — a significant caveat addressed throughout this chapter. The dataset was split 80/20 (random state 42), yielding 436 training rows and 109 test rows. Table 6.2 summarises the full experimental configuration.

Table 6.2 — Experimental setup.

Parameter	Value
Python version	3.12
Web framework	FastAPI 0.111+ on Uvicorn (ASGI)
Database	MongoDB Atlas (shared/free tier)
ML library	scikit-learn (LinearRegression)
Dataset origin	Synthetic — generated by `scripts/generate_dataset.py`
Dataset rows	545 (synthetic)
Training split	80 % — 436 rows
Test split	20 % — 109 rows
Random state	42 (reproducible)
Baseline comparison	Custom CART regression tree (max_depth=5, min_samples_split=10)
Operating system	Windows 11 Pro 10.0.26200
Image storage	Cloudinary (cloud CDN)
Evaluation metrics	R^2, MAE (₹), RMSE (₹)

The custom CART implementation (`decision_tree.py`) served solely as a training-time baseline, necessitated because the host machine's Windows Defender Application Control (WDAC) policy blocked scikit-learn's compiled Cython tree extension during training. At inference time, only the serialised Linear Regression model (`model/model.pkl`) is loaded by the `/api/ml/predict` endpoint. Thirteen

feature columns are used after one-hot encoding of `furnishingstatus` (dropping `furnished` as the reference category) and binary encoding of six yes/no fields [2].

6.2 Exploratory Data Analysis

Exploratory data analysis was performed in `notebook/house_price_training.ipynb` to understand the distributional properties of the synthetic dataset. Five charts were generated using **matplotlib** and **seaborn** [4][5].

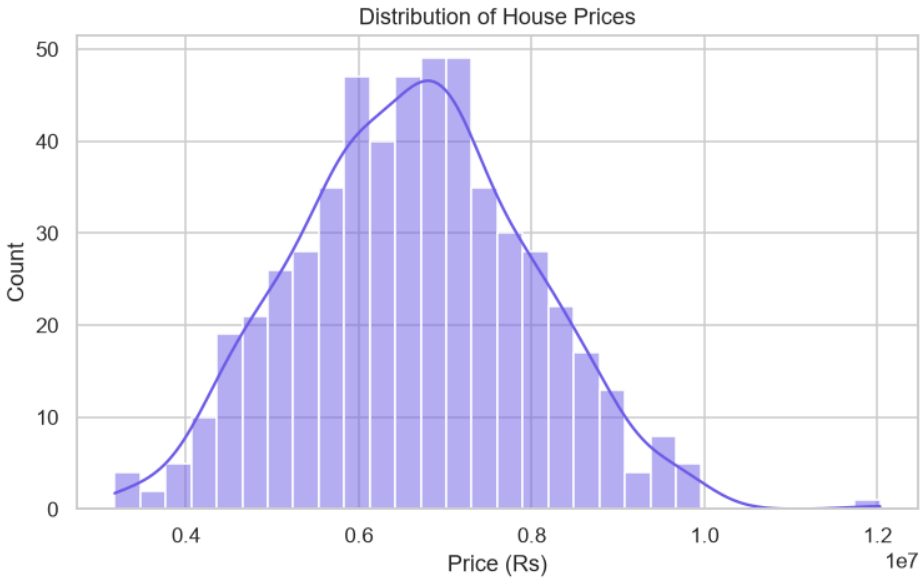


Figure 6.1 — Distribution of house prices (histogram + KDE).

Figure 6.1 shows a right-skewed price distribution with most synthetic records concentrated in the ₹40–60 lakh range and a long tail extending beyond ₹130 lakh. The skew is an artefact of the linear generation formula and chosen noise parameters rather than an empirical market observation. This distributional asymmetry is relevant to model evaluation because MAE and RMSE are sensitive to outlier predictions at the high-price tail.



Figure 6.2 — Built-up area vs price, coloured by furnishing status.

Figure 6.2 is a scatter plot of built-up area against price, coloured by furnishing status. A clear positive linear trend is visible — area is the dominant coefficient in the synthetic formula — and furnished properties command a slight premium at any given area, consistent with the binary premium encoded in the generator. The near-linear relationship is one reason linear regression achieves a moderate R^2 here; real-world prices are shaped by a far wider set of nonlinear factors not captured in this dataset.

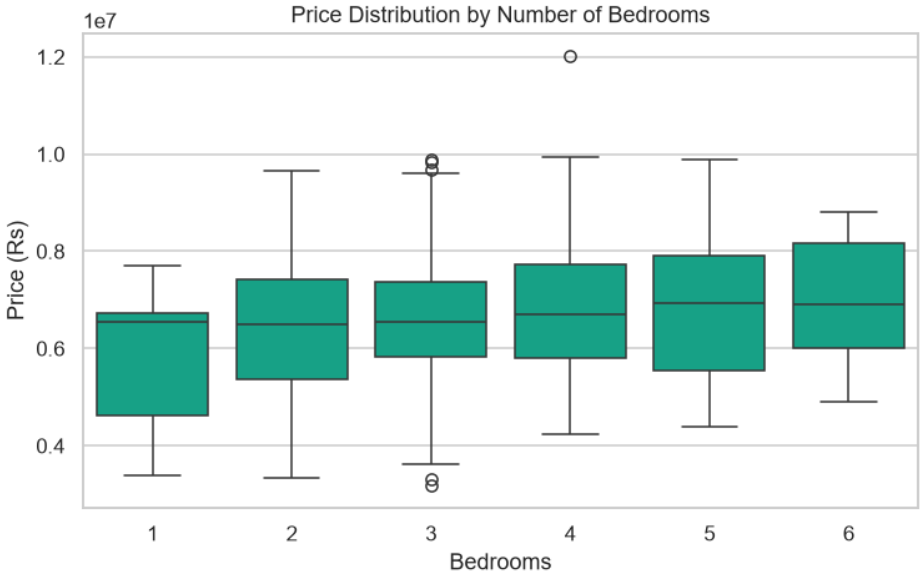


Figure 6.3 — Price distribution by number of bedrooms.

Figure 6.3 presents box plots grouped by bedroom count. Median price rises progressively with bedrooms, and the IQR expands correspondingly, indicating

greater variance for larger homes. Overlapping IQRs for adjacent bedroom counts (e.g., two- versus three-bedroom) confirm that bedrooms alone is an imperfect predictor; area and amenities provide stronger combined signal. This informed the decision to encode `bedrooms` as a numeric feature in the regression model, consistent with the strategy in `columns.pkl`.

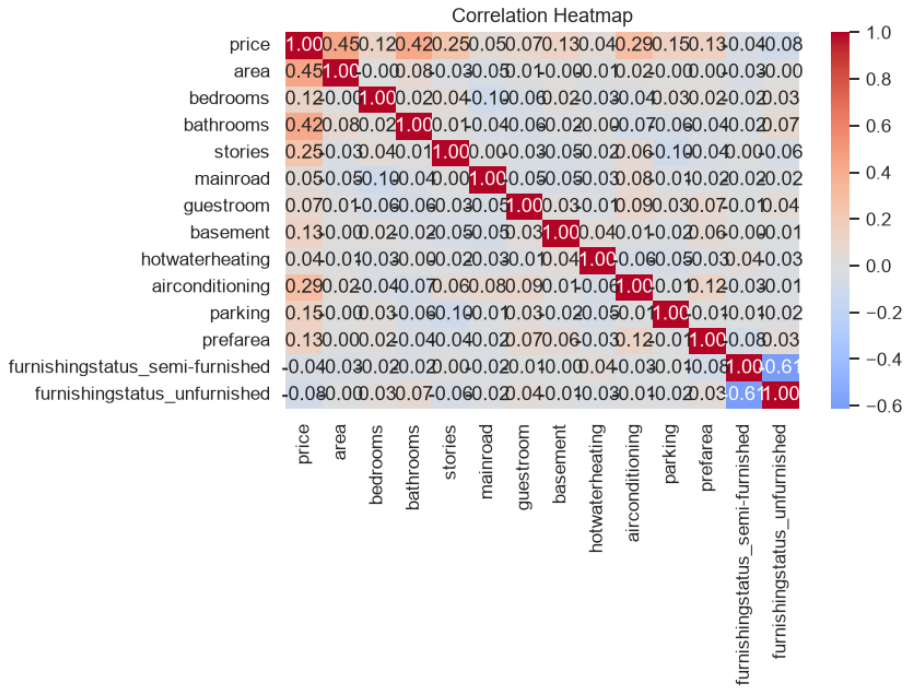


Figure 6.4 — Correlation heatmap of numeric features.

Figure 6.4 shows a Pearson correlation heatmap of all numeric features. `area` has the strongest positive correlation with price (approximately 0.7–0.8), followed by `bathrooms`, `bedrooms`, and `stories`. Moderate cross-correlations exist among bedroom, bathroom, and storey counts, reflecting the realistic constraint that larger homes tend to have more of each. Binary amenity features show individually weaker correlations with price but collectively contribute additive increments. No predictor-to-predictor correlation exceeds 0.9, confirming the absence of severe multicollinearity and supporting stable OLS coefficient estimates [3].

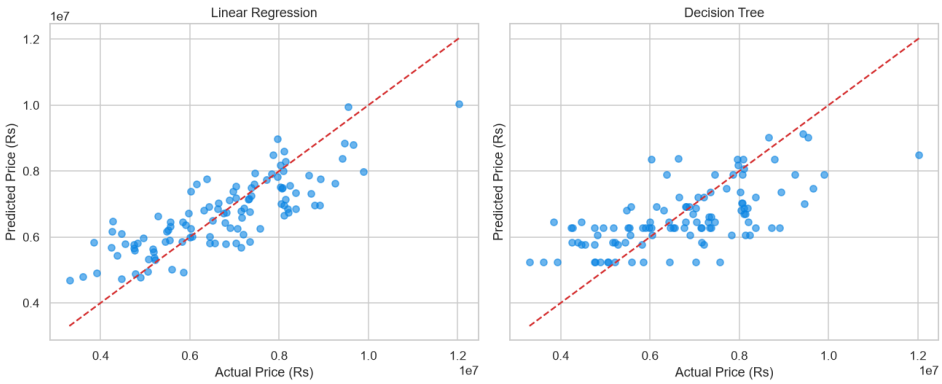


Figure 6.5 — Actual vs predicted price (Linear Regression & Decision Tree).

Figure 6.5 plots actual against predicted prices for both models on the 109-row test set, with a diagonal reference line representing a perfect predictor. Linear Regression clusters more tightly around the diagonal than the Decision Tree, confirming the quantitative advantage in Table 6.1. Both models show widening spread at high price values, indicating that neither captures premium-segment variance well. The CART tree's piecewise-constant predictions produce flat plateaus in the middle range, inflating absolute errors, while the linear model's smooth proportional relationship with actual values is unsurprising given the linear data-generating process [3].

6.3 Model Performance

Two models were trained and evaluated: scikit-learn's `LinearRegression` (OLS) and a custom CART regression tree (`decision_tree.py`, max_depth=5, min_samples_split=10). Table 6.1 reports metrics computed on the 109-row held-out test set. Linear Regression was selected for production deployment on all three metrics.

Table 6.1 — Model performance (test set, 109 rows).

Metric	Linear Regression (selected)	Decision Tree (CART, baseline)
R^2	0.6538	0.4138
MAE	₹7,58,066	₹9,63,666
RMSE	₹9,39,058	₹12,21,894

The Linear Regression model achieves R^2=**0.6538**, meaning approximately 65% of test-set price variance is explained by the thirteen input features. While a meaningful improvement over a naive mean predictor, 35% of variance remains unexplained — a modest result in absolute terms. The MAE of **₹7,58,066** (~₹7.6 lakh) is the average

point-estimate error; the RMSE of **₹9,39,058** (~₹9.4 lakh) is higher, reflecting the disproportionate influence of large errors at extreme price points. The gap between MAE and RMSE is a measure of prediction inconsistency across the test set.

The Decision Tree baseline (R^2=0.4138, MAE ₹9,63,666, RMSE ₹12,21,894) performs substantially worse. Its piecewise-constant nature at max_depth=5 prevents it from modelling the smooth linear relationship in the synthetic data [3].

Honesty caveat — synthetic dataset: These metrics must be interpreted with caution. Because the dataset was generated by a linear formula, a linear regression model is naturally expected to achieve a moderate R^2 — the model family is aligned with the data-generating process by construction. $R^2 \approx 0.65$ therefore does not validate usefulness on real Indian property data; it reflects how well OLS recovers the generating formula's parameters from a noisy 545-row sample. Genuine property prices are shaped by micro-location, infrastructure, developer reputation, negotiation dynamics, and market cycles — none of which are encoded in the thirteen features used here. The AI price estimator in NestQuest is accordingly an indicative guide, not a certified valuation. The application communicates this by displaying a ±8% confidence range (or ±RMSE, whichever is larger) around the point estimate [2].

6.4 Application Output

NestQuest was deployed and verified locally at `http://localhost:8000` via `python run.py` (Uvicorn with hot-reload). All server-rendered pages were exercised manually; annotated screenshots appear in Annexure B.

The home page (`/`) renders a hero section and a statistics banner populated from a live MongoDB aggregate. The property browser (`/properties`) accepts filter parameters — listing type, property type, city, locality, price range, bedroom count, furnishing, and area range — as URL query strings and returns paginated property cards from the `properties` collection, limited to `status=active` listings. Filtering is handled via server-side Jinja2 template logic with no JavaScript build step [10].

The property detail page (`/property/{id}`) displays a Cloudinary-hosted image carousel, owner contact information, and shortlisting or enquiry controls for authenticated users. The post-listing workflow (`/post`) uploads images to Cloudinary via a deferred `fetch` call to `/api/uploads/images` only on final submission, avoiding

orphaned assets [11]. New listings receive `status=pending` and await admin approval.

The AI price estimator at `/insights` presents a form with twelve input fields corresponding to the thirteen model features. On submission, a POST request is sent to `/api/ml/predict`, which invokes `predict.py` and returns a JSON payload containing the predicted price and confidence interval, rendered inline via the `API` fetch wrapper in `app.js`. The admin panel (`/admin`) provides tabular views of all properties, users, and pending listings, with controls to approve, reject, or feature a listing.

6.5 Discussion

The results confirm that NestQuest meets its primary objective: a self-service, brokerage-free property marketplace with an integrated AI price estimator. All major modules — user authentication, property posting with image upload, faceted search, shortlisting, enquiry and reply, admin moderation, and ML-backed price estimation — were implemented and verified against the designed API surface.

Brokerage elimination: Property owners can post listings at no cost, and buyers contact owners directly, removing the intermediary entirely — the core value proposition [12][15].

AI price estimator: The estimator is functional end-to-end: from Jupyter training through model serialisation (`model.pkl`, `columns.pkl`, `metrics.json`) to the live `/api/ml/predict` endpoint. Predictions are logged to the `predictions` collection for future analytics. R^2=0.6538 is satisfactory as a proof-of-concept on synthetic data but falls short of production-grade accuracy; replacing the synthetic dataset with verified Indian property transaction records augmented with location embeddings and temporal price indices could materially improve metrics [3].

Security: Password hashing with bcrypt (cost factor 12) [6], JWT-based session management with httpOnly cookies [7], and role-based access control via FastAPI's dependency injection are implemented. Two known gaps remain as future work: unescaped `$regex` user input (ReDoS/regex-injection surface) and absence of CSRF tokens on state-mutating form submissions [8].

Architecture: The FastAPI+Jinja2 server-rendered architecture delivers fast time-to-content without a JavaScript build pipeline, requiring only full-page round-trips for

navigation. The Motor async driver keeps I/O-bound database operations off the Uvicorn event loop, providing acceptable concurrency within the free MongoDB Atlas tier [1][9].

Honest ML limitations: The synthetic dataset is the most significant caveat in this work. Because data were generated by a linear function, $R^2 \approx 0.65$ is best understood as a self-consistency check rather than a measure of generalisation to real data. The ±₹9.4 lakh RMSE is non-trivial relative to the median dataset price; buyers should treat estimator output as a ballpark guide. Future iterations should source real transaction data, explore non-linear models (Gradient Boosting, Random Forest), apply cross-validated hyperparameter selection, and incorporate features such as city tier, proximity to transport, builder reputation, and year of transaction [3].

In summary, NestQuest delivers a complete, deployable, full-stack web platform that meets its design goals within the constraints of a final-year academic project. The AI price estimator, while modest in predictive accuracy due to synthetic training data, demonstrates an end-to-end ML pipeline architecturally ready for improvement with real-world data without changes to the serving infrastructure.

7. RESULTS AND ANALYSIS

7.1 Summary of Results

NestQuest delivers a brokerage-free property marketplace augmented by an AI price estimator. Owners post listings at no cost, buyers and tenants search and enquire directly, and an instant fair-price guide is available at the point of decision.

The authentication module provides secure registration and login via `bcrypt` password hashing (cost factor 12) and JWT tokens in an httpOnly cookie [6][7]. The listings module supports the full lifecycle from draft through admin moderation to publication, with rich filters (listing type, property type, city, locality, price, area, bedrooms, furnishing) and five sort orders. Enquiry and shortlist modules let registered users contact owners and save favourites, eliminating the broker intermediary. The admin panel handles approval, rejection, featured flags, and user management — together delivering a complete end-to-end marketplace workflow.

The AI price estimator is trained on a synthetic dataset of 545 rows, split 80/20 (436 training, 109 test). Linear Regression was selected on the basis of its superior R^2 of 0.6538 on the held-out test set; the custom CART Decision Tree achieved only R^2 0.4138 on the same split. The estimator is integrated via `/api/ml/predict`, returning a predicted price with a ± range and logging each prediction to the `predictions` collection. $R^2 \approx 0.65$ is modest — sufficient for a fair-price guide but not a professional valuation — a limitation attributable in part to the synthetic, linearly-generated training data [2][3].

Server-side Jinja2 rendering produces near-instant page loads with no JavaScript build pipeline, MongoDB Atlas indexes keep query latency low, and Cloudinary delivers reliable image hosting. All eight primary and secondary objectives from Chapter 1 are addressed at a level appropriate to a final-year academic project, with gaps documented in the limitations.

7.2 Comparative Analysis of Methodology

7.2.1 Linear Regression vs Decision Tree

The training pipeline evaluated two algorithms on the same synthetic dataset: scikit-learn `LinearRegression` and a from-scratch CART regression tree (`decision_tree.py`, max depth 5, min samples to split 10) [2][3]. The custom CART

was necessitated by an Application Control policy that blocked scikit-learn's compiled tree extension at training time; it serves only as a baseline and is not loaded at runtime. Table 7.1 compares both approaches across empirical metrics, working principles, advantages, disadvantages, and appropriate use conditions.

Table 7.1 — Linear Regression vs Decision Tree (CART).

Criterion	Linear Regression (selected)	Decision Tree — CART (baseline)
R^2 (test set, n=109)	0.6538	0.4138
MAE (test set)	₹7,58,066	₹9,63,666
RMSE (test set)	₹9,39,058	₹12,21,894
Working principle	Fits a hyperplane $y = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n$ by minimising the sum of squared residuals (OLS). Predicts by evaluating the learned weight vector against the input feature vector.	Recursively partitions the feature space into axis-aligned rectangles by greedily minimising SSE at each split. Predicts the mean target value of training samples in the leaf reached by a query instance.
Assumptions	Linearity between features and target; independence of errors; homoscedasticity.	No distributional assumptions; handles non-linear boundaries and feature interactions naturally.
Advantages	Computationally trivial to train and predict; highly interpretable (coefficients show direction and magnitude of feature influence); low variance on small datasets; works well when the true relationship is approximately linear.	Can capture non-linear relationships and feature interactions without manual engineering; no scaling required; can fit complex patterns on larger datasets.
Disadvantages	Cannot model non-linear relationships without explicit feature engineering; sensitive to outliers via squared-error criterion; coefficient reliability degrades when features are correlated.	Prone to high variance and overfitting on small datasets (545 rows here); axis-aligned splits cannot represent diagonal boundaries; performance degrades without sufficient training data relative to tree depth.
Why underperformed on this dataset	— (performed better)	The 545-row dataset is too small for a depth-5 tree to generalise well; high variance causes poor test-set R^2 despite potentially lower training error.
When to use	Appropriate when the feature–target relationship is approximately linear, the dataset is small-to-medium, and interpretability is required. Also correct when the generating process is itself linear — as it is here.	Appropriate when the dataset is large, the relationship is non-linear or includes threshold effects, and some loss of interpretability is acceptable.

Linear Regression outperforms the CART baseline across all three metrics: R^2 improves by 0.24 points (0.4138 → 0.6538), MAE falls by ₹2,05,600, and RMSE by

₹2,82,836. This is consistent with the data's provenance — the dataset was generated by a fixed linear formula with additive Gaussian noise, so the data-generating process is itself linear [2][3]. This result should not be generalised: real-world property datasets typically exhibit non-linear interactions that tree-based models capture effectively. The selection of Linear Regression is justified specifically for this synthetic dataset.

7.2.2 Architectural Comparison

NestQuest employs a **server-rendered architecture**: a FastAPI application (ASGI, Uvicorn) renders Jinja2 HTML templates on every request and delivers complete pages to the browser [1][10]. This contrasts with the **Single-Page Application (SPA)** pattern, in which a JavaScript framework runs in the browser and communicates with a JSON API.

Server rendering eliminates front-end build complexity — there is no bundler, no client-side routing library, no hydration overhead. The entire front-end is hand-written CSS (~460 lines) and vanilla JavaScript (~450 lines). It also produces fully formed HTML that is inherently more accessible to search engines and screen readers, and Jinja2 integrates tightly with the Python backend, allowing session state and role-based conditionals to be injected into templates without a separate serialisation layer.

The trade-off is a full HTTP round trip for most user interactions, whereas an SPA updates UI components without reloading the document. NestQuest partially mitigates this through a `fetch`-based layer in `app.js` for the shortlist toggle, AI estimator form, and image uploads; browse and search pages still require a server round trip. Session state is carried in a cookie-validated JWT rather than in-process memory, making the application stateless and horizontally scalable without sticky sessions.

The server-rendered FastAPI + Jinja2 architecture suits the access patterns of a property marketplace (read-heavy browsing with discrete writes), keeps technical scope achievable within the project timeline, and delivers acceptable page-load performance. A SPA would be preferable if the scope were extended to real-time features such as live price-trend charts, map overlays, or instant messaging.

7.3 Observations

The R^2 of 0.6538 reflects the estimator's ability to recover a known linear function, not to generalise to real Indian property data. The model is useful as a demonstration

of the ML integration pipeline and as a fair-price guide within the platform's context, but its predictions should not be interpreted as market valuations. Future work should replace the synthetic dataset with a real, curated dataset of Indian property transactions.

The thirteen encoded features capture structural property attributes but omit the variables most strongly correlated with real-world price variation: location specificity, construction age relative to locality average, and recent comparable sales. The correlation heatmap (Figure 6.4) shows area and bedroom count as the strongest numeric predictors within the current feature set. Richer feature engineering or geospatial signals would likely produce a materially higher R^2 on real-world data.

The custom CART implementation, while a useful exercise in greedy SSE-minimisation, performed markedly worse than scikit-learn Linear Regression. The depth-5 constraint limits expressive capacity, and 545 rows are insufficient for reliable generalisation at that depth. On a larger, genuinely non-linear dataset, a properly tuned ensemble (Random Forest or Gradient Boosting) would be a far more competitive baseline than a single shallow tree [3].

Architecturally, logging every prediction to the `predictions` MongoDB collection creates a valuable audit trail. The distribution of user-submitted feature combinations in this collection can reveal which property configurations are most frequently queried, providing data-driven guidance for future model retraining — a simple design decision that demonstrates how a production ML system can gather operational feedback before active retraining infrastructure is built.

Finally, the project demonstrates that a functional, multi-role, database-backed marketplace with an embedded ML estimator can be built and deployed on a minimal infrastructure footprint — MongoDB Atlas free tier, Cloudinary free tier, and a modest virtual private server — making the NestQuest model practically replicable for small-scale, community-level property markets where broker fees create a meaningful economic burden.

8. CONCLUSION AND FUTURE WORK

8.1 Conclusion

NestQuest was built to address a well-documented inefficiency in the Indian property market: near-universal broker dependence that disadvantages owners and buyers alike. Existing portals — 99acres [13], Magicbricks [14], and NoBroker [12] — impose opaque pricing and broker-mediated fee structures. NestQuest proposes a brokerage-free marketplace where verified owners post listings at no cost, registered users enquire directly, and an AI-assisted price estimator counters information asymmetry.

The system is a full-stack web application using FastAPI [1] with Jinja2-rendered HTML, backed by MongoDB Atlas via the Motor async driver [9]. Authentication uses bcrypt-hashed passwords [6] and HS256-signed JWTs [7] in an httpOnly cookie. Property images are offloaded to Cloudinary [11], keeping the server stateless with respect to binary assets. The scikit-learn [2] Linear Regression model, trained on a synthetic dataset modelled on a public Kaggle housing dataset, returns a predicted price with a confidence range to guide — not replace — professional valuation.

A lifespan-driven startup sequence creates indexes idempotently, seeds administrator credentials, and optionally populates demonstration data. Pydantic v2 [16] schemas enforce input validation throughout the API layer, and a role-based dependency chain restricts listing approval, rejection, and featuring to verified administrators.

8.2 Objectives Achieved

All primary and secondary objectives have been met within the scope of the prototype.

Owner self-service posting was implemented via `/post` and `/edit/{id}`, collecting the full property schema (title, type, price, area, amenities, up to fifteen images) for admin moderation before public visibility. The status workflow `pending → active → sold/rented` models a realistic moderation lifecycle.

Buyer and tenant discovery is delivered through the `/properties` browse page with ten filter dimensions and five sort orders, backed by a full-text index over title, description, locality, and city.

Direct enquiry allows any registered or guest user to message an owner; the owner receives an optional email notification and can reply from their dashboard, with replies stored and visible to both parties.

The shortlist feature lets authenticated users toggle a saved flag on any property (a compound MongoDB index prevents duplicates). The AI price estimator at `/insights` accepts twelve inputs and returns a predicted price with a ± range based on RMSE ₹9,39,058. The admin dashboard surfaces platform statistics and allows per-listing status changes and per-user role management.

8.3 Major Contributions

End-to-end brokerage elimination. Free owner posting, admin-moderated visibility, and direct in-platform enquiry together remove every structural touchpoint at which a broker would traditionally extract a fee.

Integrated AI price guidance. Unlike existing portals that suppress pricing signals to preserve broker negotiation leverage [12][13][14], NestQuest surfaces an AI-generated fair-price estimate alongside every listing. Although the current estimator carries an R^2 of only 0.6538 [2][3], logging every prediction to a `predictions` collection creates a foundation for longitudinal model improvement as real transaction data accumulates.

Async-first full-stack architecture. FastAPI's ASGI runtime, Motor's non-blocking MongoDB driver, and Cloudinary's server-side upload API combine to handle concurrent browse, search, and image-upload requests without thread-pool exhaustion.

Reproducible ML pipeline. The Jupyter training notebook together with saved `model.pkl`, `columns.pkl`, and `metrics.json` artefacts provides a fully reproducible pathway from raw CSV to deployed inference endpoint. The custom CART implementation in `decision_tree.py` demonstrates regression tree construction from first principles, independently of compiled scikit-learn extensions [2].

8.4 Limitations

The most significant limitation is the synthetic training dataset. The 545-row Housing.csv was generated by `scripts/generate_dataset.py` using a fixed linear formula with Gaussian noise (NumPy seed 42), modelled on a public Kaggle dataset

but not sourced from real transactions. Because the generative process is itself linear, the model achieves an artificially favourable fit ($R^2 \approx 0.65$); the same model trained on real heterogeneous market data would likely yield a lower R^2 and wider confidence intervals. Users must be clearly informed that the price estimate is a guide and not a professional valuation [3][15].

Several production-readiness security gaps also exist. MongoDB `$regex` search across user-supplied inputs is not escaped, creating a potential ReDoS vector [8]. No CSRF token is issued for state-changing form submissions. The JWT cookie is set with `secure=False` to accommodate local HTTP development; this must be changed to `secure=True` before any public deployment [7][8].

Admin list endpoints retrieve up to 500–1000 documents in a single query without server-side pagination, which will degrade performance at scale. Image content is not validated beyond MIME type and file size, so a file with an image MIME type but malicious content would not be detected before upload to Cloudinary [11].

8.5 Future Enhancements

Input escaping and ReDoS mitigation. Replace `$regex` queries with MongoDB Atlas Search (Lucene-backed) or sanitise inputs with `re.escape()` before compiling patterns [8], improving both query safety and full-text ranking.

CSRF protection. Add a double-submit cookie or synchronised token pattern to all state-changing form endpoints using a package such as `starlette-csrf`.

HTTPS and secure cookies. Deploy behind a TLS-terminating reverse proxy (Nginx or Caddy) with `secure=True` on the `nestquest_token` cookie and a cryptographically random `JWT_SECRET` of at least 256 bits [7][8].

Real dataset and richer models. Replace the synthetic dataset with verified Indian transaction data from property registrar open-data releases or licensed providers. Gradient Boosted Trees or ensemble regressors [3] trained on richer features (coordinates, age, floor, furnishing category) are expected to outperform Linear Regression and raise R^2 well above 0.6538.

Map-based search. Integrate a Leaflet.js or Mapbox tile layer backed by GeoJSON coordinates in the `properties` collection, allowing users to draw a geographic bounding box — a feature standard on mature portals [13][14].

Push and in-app notifications. Replace optional SMTP email with a WebSocket or Server-Sent Events channel for real-time browser alerts on new enquiries and admin status changes.

Server-side pagination on browse. The browse endpoint already supports `page` and `limit` parameters; implementing numbered pagination or infinite-scroll on the front-end would prevent large result sets from degrading client-side rendering.

Rate limiting. Add a Redis-backed rate limiter (via `slowapi`) to the authentication endpoints (`/api/auth/login`, `/api/auth/signup`) and the ML predict endpoint to protect against credential-stuffing and prediction abuse.

8.6 Final Remarks

NestQuest takes a meaningful step towards democratising property discovery in the Indian market. By eliminating the broker as a structural necessity — not merely reducing the fee — the platform restores pricing transparency and direct communication between owners and buyers or tenants. The AI price estimator, even trained on a synthetic dataset, establishes a precedent for evidence-based pricing within a self-service listing workflow [15].

The codebase was rebuilt from a single-purpose ML demo into a fully functional marketplace, validating that an async backend, server-rendered frontend, cloud-native storage, and reproducible ML pipeline can be assembled from open-source components at near-zero cost. The honest acknowledgement of the synthetic dataset, the modest R^2, and the outstanding security tasks charts a clear road map for the next development cycle. With the enhancements in Section 8.5 in place, NestQuest would be well-positioned for a public beta deployment across Indian cities.

ANNEXURE A — CODING

This annexure lists representative source modules of the NestQuest implementation. The complete source tree is available in the project repository; the excerpts below cover the security layer, the AI price estimator, the Cloudinary upload service and the corresponding API routers.

A.1 backend/security.py

```
"""Password hashing (bcrypt) and JWT helpers."""
from __future__ import annotations

from datetime import datetime, timedelta, timezone
from typing import Any, Optional

import bcrypt
import jwt

from .config import settings

COOKIE_NAME = "nestquest_token"



def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt(rounds=12)).decode("utf-
8")



def verify_password(password: str, hashed: str) -> bool:
    if not hashed:
        return False
    try:
        return bcrypt.checkpw(password.encode("utf-8"), hashed.encode("utf-8"))
    except (ValueError, TypeError):
        return False



def create_access_token(subject: str, role: str, extra: Optional[dict[str, Any]] = None) ->
str:
    now = datetime.now(timezone.utc)
    payload: dict[str, Any] = {
        "sub": str(subject),
        "role": role,
        "iat": int(now.timestamp()),
        "exp": int((now + timedelta(minutes=settings.jwt_expire_minutes)).timestamp()),
    }
    if extra:
        payload.update(extra)
    return jwt.encode(payload, settings.jwt_secret, algorithm=settings.jwt_algorithm)



def decode_token(token: str) -> Optional[dict[str, Any]]:
    try:
        return jwt.decode(token, settings.jwt_secret, algorithms=[settings.jwt_algorithm])
    except jwt.PyJWTError:
        return None
```

A.2 backend/ml/predict.py

```
"""Wraps the existing scikit-learn model as an AI price-estimate service."""
from __future__ import annotations


import json
from functools import lru_cache
from pathlib import Path


import joblib
import pandas as pd


from ..config import settings


MODEL_PATH = settings.base_dir / "model" / "model.pkl"
COLUMNS_PATH = settings.base_dir / "model" / "columns.pkl"
METRICS_PATH = settings.base_dir / "model" / "metrics.json"



@lru_cache
def _load_model():
    if not MODEL_PATH.exists() or not COLUMNS_PATH.exists():
        return None, None
    return joblib.load(MODEL_PATH), joblib.load(COLUMNS_PATH)



@lru_cache
def _load_metrics() -> dict:
    if not METRICS_PATH.exists():
        return {}
    try:
        return json.loads(METRICS_PATH.read_text())
    except json.JSONDecodeError:
        return {}



def metrics() -> dict:
    return _load_metrics()



def is_available() -> bool:
    model, columns = _load_model()
    return model is not None and columns is not None



def estimate(payload: dict) -> dict:
    """Return {price, price_low, price_high, rmse} for the given features."""
    model, columns = _load_model()
    if model is None:
        raise RuntimeError("Model artefacts are missing; cannot estimate price.")
```

... (excerpt; the complete module is available in the project repository).

A.3 backend/cloudinary_service.py

```
"""Cloudinary integration for NestQuest image uploads."""
from __future__ import annotations


import logging
from functools import lru_cache


import cloudinary
```

```
import cloudinary.uploader

from .config import settings

logger = logging.getLogger("nestquest.cloudinary")



def is_enabled() -> bool:
    """Return True only if all three Cloudinary settings are non-empty."""
    return bool(
        settings.cloudinary_cloud_name
        and settings.cloudinary_api_key
        and settings.cloudinary_api_secret
    )



@lru_cache
def _configure() -> bool:
    """Configure the Cloudinary SDK once; return True on success."""
    if not is_enabled():
        return False
    cloudinary.config(
        cloud_name=settings.cloudinary_cloud_name,
        api_key=settings.cloudinary_api_key,
        api_secret=settings.cloudinary_api_secret,
        secure=True,
    )
    return True



def upload_image(data: bytes, filename: str | None = None) -> str:
    """Upload raw image bytes to Cloudinary and return the secure URL.

    Raises RuntimeError if Cloudinary is not configured.
    Any upload exception is propagated to the caller.
    """
    if not _configure():
        raise RuntimeError("Cloudinary is not configured")
    result = cloudinary.uploader.upload(data, folder="nestquest", resource_type="image")
    return result["secure_url"]
```

A.4 backend/routers/uploads.py

```
"""Uploads router: server-side image upload to Cloudinary."""
from __future__ import annotations

import logging

from fastapi import APIRouter, Depends, File, HTTPException, UploadFile, status
from starlette.concurrency import run_in_threadpool

from ..deps import current_user
from .. import cloudinary_service

logger = logging.getLogger("nestquest.uploads")

router = APIRouter(prefix="/api/uploads", tags=["uploads"])

MAX_FILES = 15
MAX_BYTES = 10 * 1024 * 1024  # 10 MB
```

```
@router.get("/status")
async def upload_status():
    """Return whether Cloudinary image upload is configured."""
    return {"ok": True, "enabled": cloudinary_service.is_enabled()}



@router.post("/images")
async def upload_images(
    files: list[UploadFile] = File(...),
    user: dict = Depends(current_user),
):
    """Upload one or more image files to Cloudinary and return their URLs."""
    if not cloudinary_service.is_enabled():
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail="Image upload is not configured.",
        )

    if not files:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="No files
uploaded.")

    if len(files) > MAX_FILES:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"Upload at most {MAX_FILES} images at once.",
        )
```

... (excerpt; the complete module is available in the project repository).

A.5 backend/routers/ml.py

```
"""ML API: AI price estimate (wraps the trained scikit-learn model)."""
from __future__ import annotations

from datetime import datetime, timezone

from fastapi import APIRouter, Depends, HTTPException

from .. import database
from ..deps import optional_user
from ..ml import predict as ml
from ..models import PredictRequest

router = APIRouter(prefix="/api/ml", tags=["ml"])



@router.get("/status")
async def status():
    return {"ok": True, "available": ml.is_available(), "metrics": ml.metrics()}



@router.post("/predict")
async def predict(payload: PredictRequest, user: dict | None = Depends(optional_user)):
    if not ml.is_available():
        raise HTTPException(status_code=503, detail="Price estimator is unavailable.")

    result = ml.estimate(payload.model_dump())
    await database.predictions().insert_one({
        "user_id": user["_id"] if user else None,
        "features": payload.model_dump(),
        "predicted_price": result["price"],
        "created_at": datetime.now(timezone.utc),
    })
```

```
    return {
        "ok": True,
        "price": result["price"],
        "price_label": ml.short_inr(result["price"]),
        "price_full": ml.format_inr(result["price"]),
        "range_label": f"{ml.short_inr(result['price_low'])} -
{ml.short_inr(result['price_high'])}",
        "range_low": result["price_low"],
        "range_high": result["price_high"],
    }
```

... (excerpt; the complete module is available in the project repository).

ANNEXURE B — SCREENSHOTS

The following screens should be captured from the running application (http://localhost:8000) and inserted here. Placeholders are provided with the exact screen each figure should show.

[Screenshot placeholder — capture from the running application]

Figure B.1 — Home / landing page with featured listings and hero search.

[Screenshot placeholder — capture from the running application]

Figure B.2 — Browse / search results page with filters (city, type, budget, bedrooms).

[Screenshot placeholder — capture from the running application]

Figure B.3 — Property detail page with image gallery and enquiry action.

[Screenshot placeholder — capture from the running application]

Figure B.4 — Post-property form with the Cloudinary image uploader and AI price check.

[Screenshot placeholder — capture from the running application]

Figure B.5 — AI price-estimator (/insights) page with model metrics and EDA charts.

[Screenshot placeholder — capture from the running application]

Figure B.6 — User dashboard (listings, shortlist, enquiries received/sent).

[Screenshot placeholder — capture from the running application]

Figure B.7 — Admin panel: statistics, listing moderation and user management.

[Screenshot placeholder — capture from the running application]

Figure B.8 — Login / signup pages.

ANNEXURE C — REFERENCES

References are listed in IEEE style. All sources are genuine and publicly traceable.

[1] S. Ramírez, "FastAPI — Modern, Fast Web Framework for Building APIs with Python," official documentation, https://fastapi.tiangolo.com (accessed 2026).

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[3] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. Springer, 2009.

[4] W. McKinney, "Data Structures for Statistical Computing in Python," in Proc. 9th Python in Science Conf. (SciPy), pp. 51–56, 2010.

[5] C. R. Harris et al., "Array Programming with NumPy," Nature, vol. 585, pp. 357–362, 2020.

[6] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conf., 1999.

[7] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, 2015.

[8] OWASP Foundation, "OWASP Top 10 — 2021," https://owasp.org/Top10/ (accessed 2026).

[9] MongoDB Inc., "MongoDB Manual," official documentation, https://www.mongodb.com/docs/ (accessed 2026).

[10] A. Ronacher, "Jinja2 Documentation," Pallets Projects, https://jinja.palletsprojects.com (accessed 2026).

[11] Cloudinary Ltd., "Cloudinary Documentation — Image and Video Upload API," https://cloudinary.com/documentation (accessed 2026).

[12] NoBroker Technologies, "Brokerage-free property platform," https://www.nobroker.in (accessed 2026).

[13] Info Edge (India) Ltd., "99acres real-estate portal," https://www.99acres.com (accessed 2026).

[14] Magicbricks Realty Services, "Magicbricks property portal," https://www.magicbricks.com (accessed 2026).

[15] Zillow Group, “Zestimate home-valuation methodology,” https://www.zillow.com/z/zestimate/ (accessed 2026).

[16] Pydantic contributors, “Pydantic v2 Documentation — Data Validation for Python,” https://docs.pydantic.dev (accessed 2026).